

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO
CÁC HỆ THỐNG PHÂN TÁN
ĐỀ TÀI:
ỨNG DỤNG KIẾN TRÚC MICROSERVICES TRONG XÂY DỰNG HỆ THỐNG ĐẶT
VÉ SỰ KIỆN FLASH TICKET

Giảng viên hướng dẫn	Kim Ngọc Bách
Lớp	D22HTTT3
Nhóm	10
Thành viên	MSV
Nguyễn Việt Huy	B21DCCN437
Trần Anh Tú	B22DCCN749

Hà Nội, 2026

Mục lục

Mục lục.....	2
LỜI CẢM ƠN.....	6
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI.....	7
1.1. Giới thiệu đề tài.....	7
1.2. Lý do chọn đề tài.....	7
1.3. Mục tiêu của đề tài.....	8
1.4. Đối tượng sử dụng hệ thống.....	8
1.4.1. Người chơi.....	9
1.4.2. Quản trị viên hệ thống.....	9
1.5. Phạm vi chức năng của hệ thống.....	10
1.6. Kiến trúc tổng quan của hệ thống.....	10
1.7. Đóng góp của đề tài.....	11
1.8. Giới hạn của đề tài.....	11
1.9. Kết chương.....	12
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	12
2.1. Kiến trúc microservice.....	12
2.1.1. Khái niệm microservice.....	12
2.1.2. Đặc điểm của kiến trúc microservice.....	13
2.1.3. Ưu điểm và hạn chế của microservice.....	14
2.2. Giao tiếp đồng bộ bằng RESTful API.....	14
2.3. Giao tiếp bất đồng bộ bằng Message Queue.....	15
2.3.1. Khái niệm Message Queue.....	15
2.3.2. RabbitMQ trong hệ thống.....	16
2.3.3. Ưu điểm của xử lý bất đồng bộ.....	16
2.4. API Gateway và Service Discovery.....	17
2.4.1. API Gateway.....	17
2.4.2. Eureka Service Discovery.....	18

2.5. Load Balancing và Fault Tolerance.....	19
2.5.1. Ribbon Load Balancing.....	19
2.5.2. Hystrix Circuit Breaker.....	19
2.6. Spring Boot và Spring Cloud.....	20
2.6.1. Spring Boot.....	20
2.6.2. Spring Cloud.....	20
2.7. Frontend và kiểm thử hệ thống.....	20
2.7.1. Frontend HTML/JavaScript.....	20
2.7.2. Kiểm thử End-to-End bằng Cucumber.....	21
2.8. Kết chương.....	21
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG “CUỘC ĐUA PHÉP NHÂN”.....	22
3.1. Phân tích yêu cầu hệ thống.....	22
3.1.1. Yêu cầu chức năng.....	22
3.1.2. Yêu cầu phi chức năng.....	23
3.2. Kiến trúc tổng thể của hệ thống.....	24
3.2.1. Biểu đồ Use case tổng quát.....	24
3.2.2. Các kịch bản chính (Use Case Specifications).....	24
3.3. Thiết kế các microservice.....	27
3.3.1. API Gateway.....	28
3.3.2. Quản lý cấu hình dịch vụ (Configuration Management).....	28
3.3.3. Eureka Discovery Server.....	28
3.3.4. Multiplication Microservice.....	29
3.3.5. Gamification Microservice.....	29
3.3.6. Quest Microservice.....	29
3.3.7. Giao tiếp REST liên dịch vụ (Inter-service REST call handler).....	30
3.4. Thiết kế cơ sở dữ liệu.....	30
3.4.1. Nhóm dữ liệu Multiplication Service.....	30
3.4.2. Nhóm dữ liệu Gamification Service.....	31
3.4.3. Nhóm dữ liệu Quest Service.....	31
3.5. Thiết kế RESTful API.....	32
3.5.1. API cho Multiplication Service.....	32

3.5.2. API cho Gamification Service.....	33
3.5.3. API cho Quest Service.....	33
3.6. Thiết kế luồng Message Queue và Biểu đồ tuần tự (Sequence Diagram).....	34
3.6.1. Luồng Giải toán và Xuất bản Sự kiện (Event Publishing).....	34
3.6.2. Luồng Xử lý Điểm số và Huy chương Bất đồng bộ (Gamification).....	35
3.6.3. Luồng Cập nhật Tiến trình Quest và Nhận thưởng (Quest Progress & Claim Reward). 35	
3.7. Thiết kế bảo mật hệ thống.....	36
3.8. Kết chương.....	37
CHƯƠNG 4: XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG “CUỘC ĐUA PHÉP NHÂN”	37
4.1. Môi trường phát triển và công nghệ sử dụng.....	37
4.1.1. Backend Microservices.....	38
4.1.2. Frontend Web UI.....	39
4.1.3. Cơ sở dữ liệu và hạ tầng thông điệp.....	39
4.1.4. Công cụ kiểm thử và vận hành.....	39
4.2. Xây dựng Backend Microservices.....	40
4.2.1. Xây dựng Multiplication Service (social-multiplication).....	40
4.2.2. Xây dựng Gamification Service (gamification).....	41
4.2.3. Xây dựng Quest Service (quest-service).....	42
4.2.4. Xây dựng Zuul API Gateway (gateway).....	42
4.2.5. Xây dựng Eureka Service Discovery Server (service-registry).....	44
4.3. Xây dựng Frontend UI.....	44
4.3.1. Giao diện trang chủ và làm toán.....	44
4.3.2. Giao diện Bảng xếp hạng và Bảng huy chương.....	46
4.3.3. Giao diện Thử thách nhiệm vụ (Quests Dashboard).....	47
4.4. Tích hợp giao tiếp đồng bộ RESTful API qua API Gateway.....	48
4.5. Cơ chế gọi REST liên dịch vụ (Inter-service Communication).....	48
4.6. Tích hợp RabbitMQ cho xử lý bất đồng bộ (Event-Driven).....	49
4.7. Cơ chế lưu trữ và cơ sở dữ liệu nhúng H2 Database.....	50
4.8. Cơ chế xoay vòng thử thách tuần hoàn (Conveyor Board).....	51
4.9. Triển khai và vận hành hệ thống.....	53

4.10. Kết chương.....	55
CHƯƠNG 5: ĐÁNH GIÁ VÀ KẾT LUẬN.....	55
5.1. Đánh giá khả năng chịu lỗi và tính toàn vẹn hệ thống (Resilience).....	55
5.2. Ưu điểm của hệ thống.....	57
5.3. Hạn chế còn tồn tại.....	57
5.4. Hướng phát triển trong tương lai.....	58
5.5. Kết luận.....	59
THAM KHẢO.....	59

LỜI CẢM ƠN

Trong quá trình thực hiện báo cáo và xây dựng hệ thống game luyện tập phép nhân theo kiến trúc microservice, nhóm chúng em đã nhận được nhiều sự hướng dẫn, hỗ trợ và tạo điều kiện từ giảng viên, nhà trường cũng như các nguồn tài liệu học thuật liên quan đến lĩnh vực hệ thống phân tán và phát triển phần mềm hiện đại.

Trước hết, nhóm chúng em xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn học phần đã truyền đạt những kiến thức nền tảng quan trọng về kiến trúc microservice, RESTful API, message queue và các công nghệ liên quan. Những kiến thức này là cơ sở giúp nhóm có thể phân tích bài toán, lựa chọn mô hình phù hợp và triển khai hệ thống theo đúng định hướng của học phần.

Nhóm chúng em cũng xin cảm ơn nhà trường và khoa chuyên ngành đã tạo môi trường học tập thuận lợi, giúp sinh viên có cơ hội tiếp cận các công nghệ thực tế như Spring Boot, Spring Cloud, RabbitMQ và các kỹ thuật phát triển hệ thống phân tán hiện đại. Thông qua quá trình thực hiện đề tài, nhóm đã có cơ hội củng cố kiến thức chuyên môn, đồng thời hiểu rõ hơn những khó khăn thực tế khi xây dựng một hệ thống gồm nhiều service độc lập và giao tiếp với nhau theo cả cơ chế đồng bộ lẫn bất đồng bộ.

Trong quá trình thực hiện, mặc dù nhóm đã cố gắng hoàn thiện hệ thống và báo cáo một cách tốt nhất, nhưng do thời gian thực hiện và kinh nghiệm thực tế còn hạn chế nên đề tài khó tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được những ý kiến đóng góp và nhận xét từ giảng viên để có thể tiếp tục hoàn thiện hệ thống cũng như rút kinh nghiệm cho các dự án sau này.

Cuối cùng, nhóm xin chân thành cảm ơn tất cả những cá nhân và tập thể đã hỗ trợ nhóm trong suốt quá trình học tập và thực hiện đề tài.

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Giới thiệu đề tài

Trong những năm gần đây, kiến trúc microservice ngày càng được sử dụng rộng rãi trong phát triển các hệ thống phần mềm hiện đại nhờ khả năng mở rộng, triển khai độc lập và phân tách nghiệp vụ rõ ràng. Thay vì xây dựng toàn bộ hệ thống dưới dạng monolithic, kiến trúc microservice chia hệ thống thành nhiều service nhỏ, mỗi service đảm nhiệm một chức năng riêng và giao tiếp với nhau thông qua API hoặc message queue.

Bên cạnh đó, các ứng dụng học tập theo hướng gamification cũng ngày càng phổ biến trong giáo dục. Gamification giúp tăng tính tương tác và động lực học tập bằng cách bổ sung các yếu tố như điểm số, bảng xếp hạng hoặc phần thưởng vào quá trình học.

Từ các xu hướng trên, đề tài xây dựng một hệ thống game luyện tập phép nhân theo kiến trúc microservice nhằm minh họa cách tổ chức và vận hành các service độc lập trong một hệ thống phân tán. Hệ thống không chỉ hỗ trợ người dùng luyện tập phép nhân mà còn tích hợp cơ chế tính điểm và bảng xếp hạng để tăng tính tương tác.

Ngoài mục tiêu chức năng, hệ thống còn được sử dụng để thực hành các pattern phổ biến trong microservice như Service Discovery, API Gateway, Load Balancing, Circuit Breaker và Event-driven Architecture. Hệ thống kết hợp giao tiếp đồng bộ thông qua REST API và giao tiếp bất đồng bộ thông qua RabbitMQ nhằm mô phỏng cách các hệ thống hiện đại xử lý request và trao đổi dữ liệu giữa các service.

1.2. Lý do chọn đề tài

Kiến trúc microservice hiện đang là một trong những hướng phát triển phổ biến trong các hệ thống phân tán hiện đại. Nhiều nền tảng lớn như Netflix, Amazon hoặc Spotify đều sử dụng microservice để tăng khả năng mở rộng và giảm phụ thuộc giữa các thành phần của hệ thống. Tuy nhiên, việc tiếp cận microservice đối với sinh viên hoặc người mới học thường gặp khó khăn do hệ thống thực tế có quy mô lớn và phức tạp.

Vì vậy, việc xây dựng một hệ thống nhỏ nhưng vẫn thể hiện được các thành phần cốt lõi của microservice là cần thiết. Ứng dụng game luyện tập phép nhân là một bài toán phù hợp vì có nghiệp vụ đơn giản, dễ triển khai nhưng vẫn có thể tích hợp nhiều kỹ thuật quan trọng như REST API, service discovery, API Gateway và xử lý sự kiện bất đồng bộ.

Ngoài ra, việc kết hợp gamification giúp hệ thống có tính tương tác cao hơn so với các ứng dụng CRUD thông thường. Thông qua điểm số và bảng xếp hạng, người dùng có thêm động lực trong quá trình luyện tập phép tính.

Đề tài cũng giúp người thực hiện tiếp cận với các công nghệ phổ biến trong hệ sinh thái Spring như Spring Boot, Spring Cloud, RabbitMQ và Maven. Đây đều là các công nghệ có giá trị thực tiễn cao trong phát triển backend hiện đại.

1.3. Mục tiêu của đề tài

Mục tiêu chính của đề tài là xây dựng một hệ thống game luyện tập phép nhân theo kiến trúc microservice, trong đó các service được tách theo miền chức năng và phối hợp với nhau thông qua REST API và RabbitMQ.

Bảng 1. Mục tiêu của đề tài

STT	Mục tiêu	Nội dung
1	Xây dựng ứng dụng luyện tập phép nhân	Cho phép người dùng nhận phép nhân ngẫu nhiên và gửi kết quả
2	Áp dụng kiến trúc microservice	Tách hệ thống thành nhiều service độc lập
3	Thiết kế RESTful API	Cung cấp API cho frontend và giao tiếp giữa các service
4	Tích hợp message queue	Sử dụng RabbitMQ cho luồng xử lý bất đồng bộ
5	Áp dụng service discovery	Sử dụng Eureka Server để quản lý service
6	Xây dựng API Gateway	Sử dụng Zuul để định tuyến request
7	Hỗ trợ load balancing và fault tolerance	Kết hợp Ribbon và Hystrix
8	Xây dựng cơ chế gamification	Tính điểm và hiển thị leaderboard
9	Kiểm thử hệ thống	Thực hiện kiểm thử end-to-end bằng Cucumber

1.4. Đối tượng sử dụng hệ thống

Hệ thống được thiết kế cho hai nhóm người dùng chính gồm người chơi và quản trị viên hệ thống. Mỗi nhóm có nhu cầu và chức năng sử dụng khác nhau.

Bảng 2. Các nhóm người dùng chính của hệ thống

Đối tượng	Vai trò chính	Nhu cầu sử dụng
-----------	---------------	-----------------

Người chơi	Luyện tập phép nhân	Nhận bài toán, gửi kết quả, xem điểm và bảng xếp hạng
Quản trị viên	Quản lý hệ thống	Theo dõi trạng thái service và hoạt động hệ thống

1.4.1. Người chơi

Người chơi là nhóm người dùng chính của hệ thống. Họ có thể truy cập giao diện web để luyện tập các phép tính nhân ngẫu nhiên do hệ thống sinh ra.

Người chơi nhập kết quả và gửi lên hệ thống để kiểm tra. Nếu câu trả lời đúng, hệ thống sẽ cập nhật điểm số thông qua service gamification. Người chơi cũng có thể xem bảng xếp hạng hoặc thống kê kết quả đã thực hiện.

Các chức năng chính của người chơi gồm:

- Nhận phép nhân ngẫu nhiên.
- Gửi kết quả phép tính.
- Xem kết quả đúng hoặc sai.
- Xem điểm số cá nhân.
- Xem bảng xếp hạng leaderboard.
- Tra cứu lịch sử kết quả đã thực hiện.

1.4.2. Quản trị viên hệ thống

Quản trị viên là nhóm người dùng có nhiệm vụ theo dõi hoạt động cơ bản của hệ thống. Trong phạm vi đề tài, chức năng quản trị chủ yếu phục vụ mục tiêu học tập và giám sát hệ thống microservice.

Quản trị viên có thể theo dõi trạng thái hoạt động của các service thông qua Eureka Server và kiểm tra khả năng giao tiếp giữa các service trong hệ thống.

Các chức năng chính của quản trị viên gồm:

- Theo dõi trạng thái các service.
- Kiểm tra đăng ký service trên Eureka.
- Theo dõi hoạt động của RabbitMQ.
- Kiểm tra gateway routing.

- Theo dõi hoạt động cơ bản của hệ thống.

1.5. Phạm vi chức năng của hệ thống

Hệ thống tập trung vào các chức năng chính phục vụ quá trình luyện tập phép nhân và minh họa kiến trúc microservice.

Bảng 3. Phạm vi chức năng của hệ thống

Nhóm chức năng	Mô tả
Tạo phép nhân	Sinh phép nhân ngẫu nhiên
Kiểm tra kết quả	Nhận và kiểm tra đáp án người dùng
Quản lý attempt	Lưu kết quả thực hiện
Gamification	Tính điểm và cập nhật leaderboard
REST API	Giao tiếp giữa frontend và backend
Message Queue	Publish và consume event qua RabbitMQ
Service Discovery	Quản lý service bằng Eureka
API Gateway	Định tuyến request qua Zuul
Load Balancing	Phân phối request bằng Ribbon
Fault Tolerance	Xử lý lỗi bằng Hystrix
Frontend	Giao diện HTML/JavaScript
Kiểm thử E2E	Kiểm thử toàn bộ hệ thống bằng Cucumber

1.6. Kiến trúc tổng quan của hệ thống

Hệ thống được xây dựng theo kiến trúc microservice với nhiều module độc lập.

Module social-multiplication đóng vai trò backend chính, phụ trách tạo phép nhân, nhận kết quả và publish sự kiện vào RabbitMQ.

Module gamification phụ trách xử lý điểm số và bảng xếp hạng. Service này lắng nghe sự kiện từ RabbitMQ để cập nhật dữ liệu mà không làm ảnh hưởng đến request gốc.

Module service-registry sử dụng Eureka Server để hỗ trợ service discovery. Các service sẽ đăng ký với Eureka để các thành phần khác có thể tìm thấy nhau thông qua tên service thay vì địa chỉ cố định.

Module gateway sử dụng Zuul API Gateway với prefix /api. Gateway tiếp nhận request từ frontend và định tuyến đến các service tương ứng. Gateway cũng tích hợp Ribbon và Hystrix để hỗ trợ load balancing và fault tolerance.

Module ui là giao diện web tĩnh xây dựng bằng HTML và JavaScript, chạy bằng Jetty.

Ngoài ra, hệ thống còn có module tests_e2e sử dụng Cucumber để kiểm thử end-to-end.

Hệ thống kết hợp hai kiểu giao tiếp:

- Giao tiếp đồng bộ thông qua REST API giữa frontend, gateway và backend services.
- Giao tiếp bất đồng bộ thông qua RabbitMQ giữa các service backend.

1.7. Đóng góp của đề tài

Đề tài có một số đóng góp chính như sau:

Thứ nhất, đề tài xây dựng được một hệ thống game luyện tập phép nhân có cơ chế gamification với điểm số và bảng xếp hạng.

Thứ hai, hệ thống áp dụng kiến trúc microservice với nhiều service độc lập như social-multiplication service, gamification service, Eureka Server và API Gateway.

Thứ ba, đề tài kết hợp giao tiếp đồng bộ và bất đồng bộ trong cùng một hệ thống. REST API được sử dụng cho các request trực tiếp, trong khi RabbitMQ được sử dụng để xử lý các sự kiện bất đồng bộ.

Thứ tư, hệ thống áp dụng nhiều pattern quan trọng trong microservice như Service Discovery, API Gateway, Load Balancing, Circuit Breaker và Event-driven Architecture.

Thứ năm, đề tài sử dụng các công nghệ phổ biến trong hệ sinh thái Spring như Spring Boot, Spring Cloud, Spring MVC và Spring AMQP, giúp tăng giá trị học tập và thực tiễn của hệ thống.

1.8. Giới hạn của đề tài

Mặc dù hệ thống đã triển khai được nhiều thành phần quan trọng của kiến trúc microservice, đề tài vẫn tồn tại một số giới hạn nhất định.

Thứ nhất, hệ thống chủ yếu phục vụ mục tiêu học tập và minh họa kiến trúc microservice nên nghiệp vụ còn đơn giản. Chức năng gamification mới dừng ở mức tính điểm và bảng xếp hạng cơ bản.

Thứ hai, hệ thống chưa triển khai đầy đủ các cơ chế production-level như centralized logging, distributed tracing, monitoring dashboard hoặc autoscaling.

Thứ ba, hệ thống chưa tích hợp cơ chế xác thực và phân quyền người dùng. Trong phạm vi đề tài, người chơi có thể sử dụng hệ thống mà không cần đăng nhập.

Thứ tư, một số công nghệ như Zuul, Ribbon và Hystrix hiện nay đã dần được thay thế bởi các giải pháp mới hơn trong hệ sinh thái Spring Cloud. Tuy nhiên, các công nghệ này vẫn phù hợp để minh họa các pattern nền tảng của microservice.

1.9. Kết chương

Chương 1 đã trình bày tổng quan về hệ thống game luyện tập phép nhân theo kiến trúc microservice, bao gồm bối cảnh lựa chọn đề tài, mục tiêu xây dựng hệ thống, các nhóm người dùng chính, phạm vi chức năng, kiến trúc tổng quan, đóng góp và giới hạn của đề tài.

Hệ thống được định hướng là một nền tảng học tập đơn giản nhưng có khả năng minh họa rõ các thành phần quan trọng trong kiến trúc microservice như API Gateway, Service Discovery, RESTful API, RabbitMQ và Event-driven Architecture. Đây là cơ sở phù hợp để nghiên cứu, triển khai và đánh giá các kỹ thuật trong phát triển hệ thống phân tán hiện đại.

Các nội dung lý thuyết liên quan đến kiến trúc microservice, RESTful API, RabbitMQ, API Gateway, Eureka Server và các công nghệ sử dụng trong hệ thống sẽ được trình bày trong Chương 2.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Kiến trúc microservice

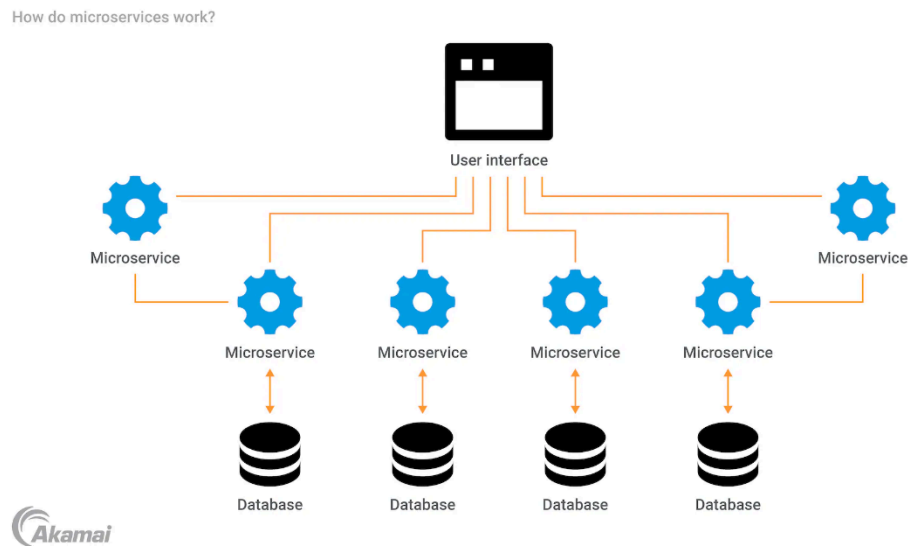
2.1.1. Khái niệm microservice

Microservice là kiến trúc phần mềm trong đó hệ thống được chia thành nhiều service nhỏ, độc lập và tập trung vào từng miền chức năng cụ thể. Mỗi service có thể được phát triển, triển khai và mở rộng riêng mà không phụ thuộc hoàn toàn vào các service khác.

Trong hệ thống game luyện tập phép nhân, kiến trúc microservice được áp dụng bằng cách tách hệ thống thành nhiều thành phần như: social-multiplication, gamification, gateway, service-registry và ui. Mỗi service đảm nhiệm một vai trò riêng thay vì gom toàn bộ chức năng vào một ứng dụng monolithic.

Ví dụ, social-multiplication xử lý nghiệp vụ tạo phép nhân và kiểm tra kết quả; gamification quản lý điểm số và bảng xếp hạng; gateway xử lý định tuyến request; còn service-registry quản lý service discovery.

Cách phân chia này giúp hệ thống dễ mở rộng, dễ bảo trì và phù hợp với mô hình hệ thống phân tán hiện đại.



2.1.2. Đặc điểm của kiến trúc microservice

Kiến trúc microservice có một số đặc điểm quan trọng sau:

Bảng 4. Đặc điểm của kiến trúc microservice trong hệ thống

Đặc điểm	Mô tả	Hệ thống game phép nhân
Tách biệt nghiệp vụ	Mỗi service phụ trách một miền chức năng riêng	social-multiplication, gamification
Triển khai độc lập	Service có thể chạy và cập nhật riêng	Mỗi service Spring Boot chạy ở port riêng
Giao tiếp qua API	Các service giao tiếp bằng REST API hoặc message queue	REST API và RabbitMQ
Service discovery	Các service tìm thấy nhau thông qua registry	Eureka Server
Dễ mở rộng	Có thể scale từng service riêng	Có thể scale gamification độc lập
Chịu lỗi tốt hơn	Lỗi một service không làm sập toàn bộ hệ thống	Hystrix circuit breaker

2.1.3. Ưu điểm và hạn chế của microservice

Kiến trúc microservice phù hợp với các hệ thống có nhiều chức năng độc lập và yêu cầu mở rộng linh hoạt. Đối với hệ thống game luyện tập phép nhân, microservice giúp tách riêng phần xử lý phép tính và phần gamification.

Ưu điểm chính gồm:

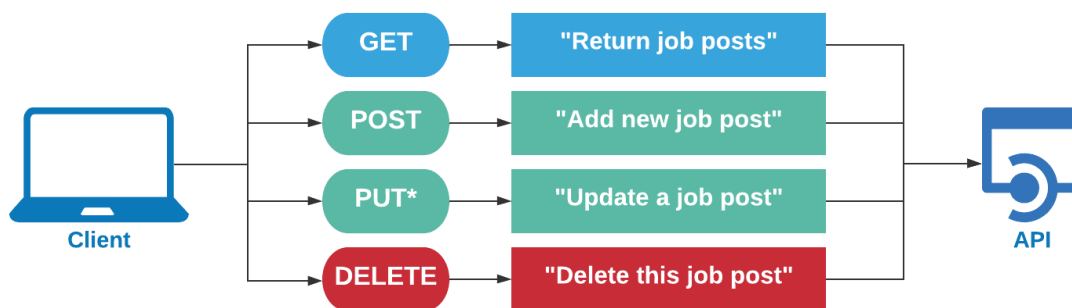
- Dễ mở rộng từng service theo nhu cầu.
- Dễ bảo trì do mỗi service có chức năng rõ ràng.
- Có thể triển khai và cập nhật độc lập.
- Giảm ảnh hưởng dây chuyền khi một service gặp lỗi.
- Phù hợp với xử lý bất đồng bộ bằng RabbitMQ.

Tuy nhiên, microservice cũng có một số hạn chế:

- Cấu hình và triển khai phức tạp hơn monolithic.
- Cần xử lý giao tiếp giữa các service.
- Debug khó hơn do request có thể đi qua nhiều service.
- Cần cơ chế monitoring và logging phù hợp.
- Phát sinh độ trễ mạng giữa các service.

2.2. Giao tiếp đồng bộ bằng RESTful API

RESTful API là phương thức giao tiếp phổ biến trong các hệ thống web và microservice. REST sử dụng các phương thức HTTP như GET, POST, PUT và DELETE để thao tác với tài nguyên.



Trong hệ thống game phép nhân, frontend không gọi trực tiếp các service backend mà gửi request đến API Gateway. Gateway sau đó định tuyến request đến service phù hợp.

Bảng 5. Một số RESTful API chính trong hệ thống

Nhóm API	Endpoint minh họa	Service xử lý	Ghi chú
Tạo phép nhân	GET /multiplications/random	social-multiplication	Trả về một phép nhân ngẫu nhiên
Gửi kết quả	POST /results	social-multiplication	Gửi đáp án của người chơi
Tra cứu kết quả theo người dùng	GET /results?alias={alias}	social-multiplication	alias là tham số bắt buộc
Tra cứu kết quả theo ID	GET /results/{resultId}	social-multiplication	Dùng trong luồng tích điểm giữa các service
Xem leaderboard	GET /leaders	gamification	Trả về bảng xếp hạng hiện tại
Xem điểm/thống kê người dùng	GET /stats?userId={userId}	gamification	userId là tham số bắt buộc
Xem điểm theo lần làm bài	GET /scores/{attemptId}	gamification	Truy vấn điểm của một attempt cụ thể

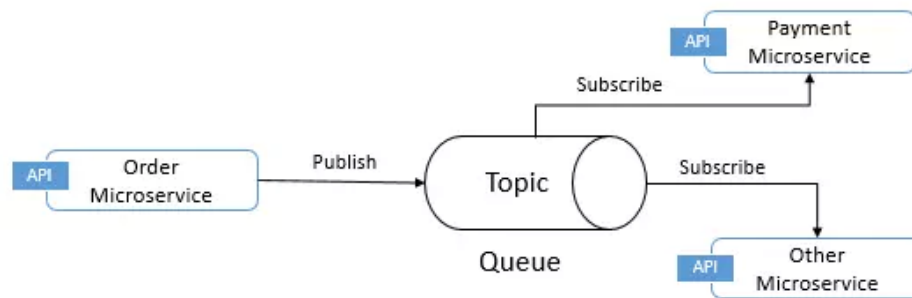
RESTful API phù hợp với các request cần phản hồi ngay, chẳng hạn như tạo phép nhân hoặc kiểm tra kết quả. Tuy nhiên, đối với các tác vụ không cần phản hồi trực tiếp như cập nhật điểm số, hệ thống sử dụng RabbitMQ để xử lý bất đồng bộ.

2.3. Giao tiếp bất đồng bộ bằng Message Queue

2.3.1. Khái niệm Message Queue

Message Queue là cơ chế giao tiếp bất đồng bộ giữa các thành phần trong hệ thống. Thay vì service A gọi trực tiếp service B và chờ phản hồi, service A có thể gửi message vào queue để service B xử lý sau.

Cách tiếp cận này giúp giảm phụ thuộc trực tiếp giữa các service và tăng khả năng mở rộng của hệ thống.



2.3.2. RabbitMQ trong hệ thống

RabbitMQ là message broker được sử dụng trong hệ thống để truyền message giữa các service.

Trong hệ thống game phép nhân, khi người dùng gửi kết quả phép tính, social-multiplication sẽ kiểm tra đáp án và publish sự kiện MultiplicationSolvedEvent vào RabbitMQ.

Service gamification lắng nghe sự kiện này thông qua `@RabbitListener`. Khi nhận được event, service sẽ cập nhật điểm số và leaderboard.

Luồng xử lý bất đồng bộ này giúp việc tính điểm không làm chậm request gốc từ người dùng.

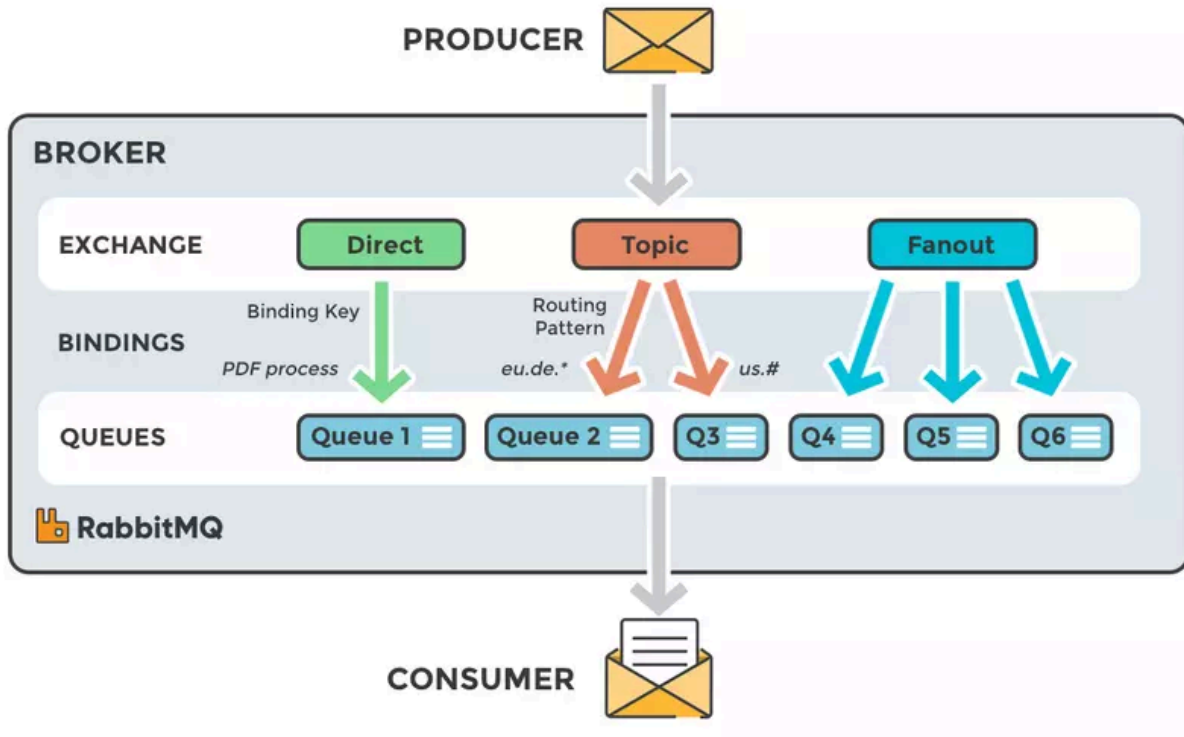
RabbitMQ trong hệ thống được dùng cho các luồng chính như:

- Publish MultiplicationSolvedEvent.
- Consumer cập nhật điểm số.
- Đồng bộ leaderboard.
- Xử lý gamification độc lập với request chính.

2.3.3. Ưu điểm của xử lý bất đồng bộ

Việc sử dụng RabbitMQ mang lại một số lợi ích:

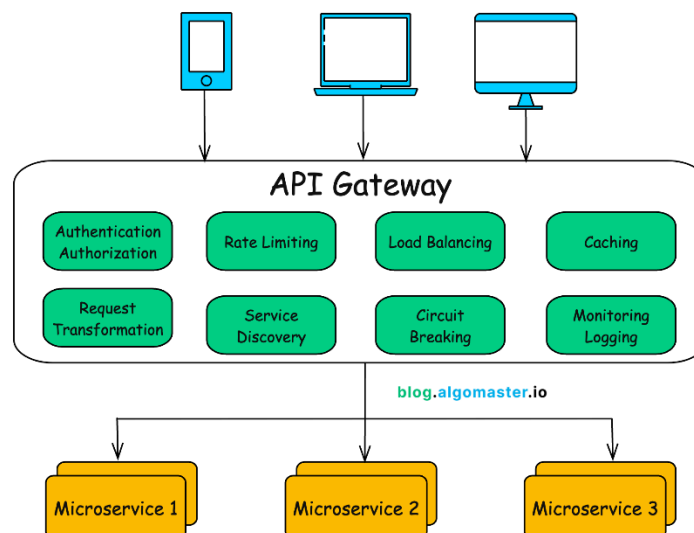
- Giảm phụ thuộc trực tiếp giữa các service.
- Tăng khả năng mở rộng hệ thống.
- Giảm thời gian phản hồi request chính.
- Hỗ trợ xử lý song song.
- Giúp hệ thống ổn định hơn khi tải tăng cao.



2.4. API Gateway và Service Discovery

2.4.1. API Gateway

API Gateway là điểm vào duy nhất của frontend khi gọi đến backend service. Thay vì frontend phải biết địa chỉ của từng service, frontend chỉ cần gửi request đến gateway.



Trong hệ thống, gateway sử dụng Zuul API Gateway với prefix /api. Gateway chịu trách nhiệm định tuyến request đến các service tương ứng.

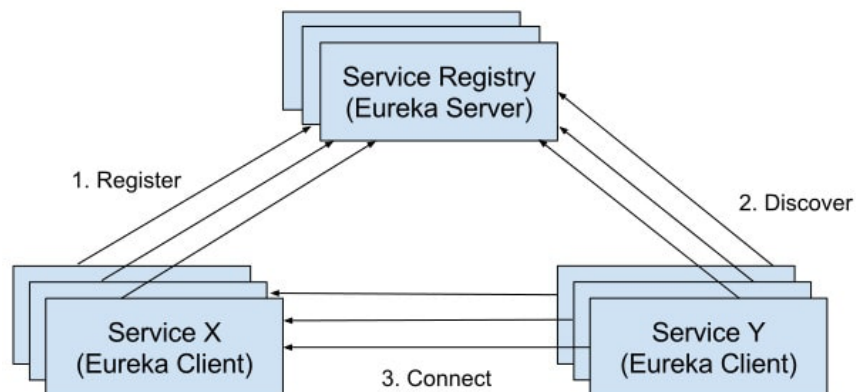
Bảng 6. Một số route chính của Gateway

Đường dẫn qua Gateway	Service đích	Ghi chú
/api/multiplications/**	social-multiplication (serviceId: multiplication)	API tạo phép nhân, lấy bài toán
/api/results/**	social-multiplication (serviceId: multiplication)	Gửi và tra cứu kết quả
/api/users/**	social-multiplication (serviceId: multiplication)	Tra cứu thông tin user
/api/leaders/**	gamification	Bảng xếp hạng
/api/scores/**	gamification	Điểm theo attempt
/api/stats/**	gamification	Thống kê điểm theo user

Ngoài định tuyến request, gateway còn tích hợp Ribbon và Hystrix để hỗ trợ load balancing và fault tolerance.

2.4.2. Eureka Service Discovery

Trong hệ thống microservice, địa chỉ service có thể thay đổi khi triển khai hoặc scale nhiều instance. Eureka Server giúp các service đăng ký và tìm thấy nhau thông qua tên service thay vì địa chỉ cố định.



Trong hệ thống này, service-registry sử dụng Eureka Server để quản lý các service. Khi một service khởi động, service sẽ đăng ký với Eureka. Gateway và các service khác có thể tìm kiếm service thông qua tên logic.

Ví dụ:

- social-multiplication
- gamification
- gateway

Cách tiếp cận này giúp hệ thống linh hoạt hơn khi triển khai nhiều instance.

2.5. Load Balancing và Fault Tolerance

2.5.1. Ribbon Load Balancing

Ribbon là thư viện hỗ trợ client-side load balancing trong Spring Cloud. Ribbon giúp phân phối request giữa nhiều instance của cùng một service.

Ví dụ, nếu triển khai nhiều instance của gamification, Ribbon có thể phân phối request giữa các instance để giảm tải cho hệ thống.

Load balancing giúp:

- Tăng khả năng chịu tải.
- Phân phối request đồng đều.
- Giảm nguy cơ quá tải một service.

2.5.2. Hystrix Circuit Breaker

Hystrix là thư viện hỗ trợ fault tolerance và circuit breaker.

Circuit breaker giúp hệ thống tránh gửi request liên tục đến một service đang gặp lỗi. Khi số lượng lỗi vượt ngưỡng, Hystrix sẽ mở circuit và trả fallback thay vì tiếp tục gọi service lỗi.

Trong hệ thống game phép nhân, Hystrix được tích hợp tại gateway để tăng khả năng chịu lỗi khi backend service phản hồi chậm hoặc không hoạt động.

Lợi ích của circuit breaker gồm:

- Giảm nguy cơ lan truyền lỗi.
- Tăng tính ổn định của hệ thống.
- Tránh timeout kéo dài.

- Cải thiện trải nghiệm người dùng.

2.6. Spring Boot và Spring Cloud

2.6.1. Spring Boot

Spring Boot là framework được sử dụng để xây dựng các backend service trong hệ thống.

Spring Boot giúp giảm cấu hình thủ công và hỗ trợ triển khai nhanh các ứng dụng Java theo mô hình standalone.

Các service như social-multiplication, gamification, gateway và service-registry đều được xây dựng bằng Spring Boot.

Spring Boot hỗ trợ:

- REST API với Spring MVC.
- Dependency Injection.
- Embedded server.
- Tích hợp RabbitMQ.
- Quản lý dependency bằng Maven.

2.6.2. Spring Cloud

Spring Cloud cung cấp các công cụ hỗ trợ xây dựng hệ thống microservice.

Trong hệ thống này, Spring Cloud được sử dụng cho:

- Eureka Service Discovery.
- Zuul API Gateway.
- Ribbon Load Balancing.
- Hystrix Circuit Breaker.

Spring Cloud giúp việc xây dựng microservice trở nên đơn giản hơn và hỗ trợ tích hợp tốt với Spring Boot.

2.7. Frontend và kiểm thử hệ thống

2.7.1. Frontend HTML/JavaScript

Frontend của hệ thống được xây dựng bằng HTML và JavaScript, chạy bằng Jetty.

Frontend có nhiệm vụ:

- Hiển thị phép nhân.
- Nhận kết quả người dùng nhập.
- Gửi request đến backend thông qua gateway.
- Hiển thị điểm số và leaderboard.

Giao diện được thiết kế đơn giản nhằm tập trung vào việc minh họa luồng hoạt động của hệ thống microservice.

2.7.2. Kiểm thử End-to-End bằng Cucumber

Hệ thống sử dụng Cucumber để thực hiện kiểm thử end-to-end.

Kiểm thử end-to-end giúp đảm bảo toàn bộ hệ thống hoạt động đúng từ frontend, gateway, backend service đến RabbitMQ.

Các bài kiểm thử tập trung vào:

- Tạo phép nhân.
- Gửi kết quả.
- Kiểm tra cập nhật điểm số.
- Kiểm tra leaderboard.
- Kiểm tra giao tiếp giữa các service.

2.8. Kết chương

Chương 2 đã trình bày các cơ sở lý thuyết và công nghệ chính được sử dụng trong hệ thống game luyện tập phép nhân theo kiến trúc microservice.

Nội dung bao gồm kiến trúc microservice, RESTful API, RabbitMQ, API Gateway, Eureka Service Discovery, Ribbon, Hystrix, Spring Boot, Spring Cloud, frontend HTML/JavaScript và kiểm thử end-to-end bằng Cucumber.

Các nền tảng lý thuyết và công nghệ này là cơ sở để triển khai hệ thống ở các chương tiếp theo, đồng thời giúp minh họa các pattern quan trọng trong phát triển hệ thống phân tán hiện đại

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG “CUỘC ĐUA PHÉP NHÂN”

3.1. Phân tích yêu cầu hệ thống

3.1.1. Yêu cầu chức năng

Hệ thống trò chơi giáo dục toán học “Cuộc Đua Phép Nhân” được thiết kế nhằm phục vụ hai nhóm đối tượng chính: **Học sinh (Người chơi - Player)** và **Hệ thống tự động (System)**. Các yêu cầu chức năng cốt lõi được đặc tả chi tiết dưới đây:

Bảng 8. Đặc tả các yêu cầu chức năng của hệ thống

Nhóm đối tượng	Yêu cầu chức năng	Mô tả chi tiết
Học sinh (Player)	1. Đăng ký biệt danh (Alias)	Người chơi nhập biệt danh đại diện khi bắt đầu tham gia trò chơi mà không cần mật khẩu phức tạp.
	2. Làm bài tập tính nhẩm	Tiếp nhận câu hỏi phép nhân ngẫu nhiên, điền kết quả và gửi đáp án lên hệ thống để chấm điểm.
	3. Theo dõi bảng điểm & huy chương	Xem điểm tích lũy hiện tại và danh sách các huy chương danh dự đã đạt được.
	4. Theo dõi bảng xếp hạng (Leaderboard)	Xem danh sách Top 10 người chơi xuất sắc nhất toàn hệ thống thời gian thực.
	5. Theo dõi tiến trình thử thách (Quest)	Theo dõi 3 nhiệm vụ active tương ứng với 3 cấp độ (Dễ, Trung bình, Khó) hiển thị trên màn hình.
	6. Nhận phần thưởng hoàn thành Quest	Bấm nút nhận thưởng điểm khi thanh tiến độ của Quest active đạt 100% để cộng điểm trực tiếp lên Leaderboard.

Hệ thống (System)	1. Tự động sinh câu hỏi ngẫu nhiên	Tự động phát sinh ngẫu nhiên hai thừa số trong khoảng từ 11 đến 99 khi có yêu cầu làm toán.
	2. Tự động chấm điểm và kiểm tra kết quả	Đối chiếu đáp án của người chơi, cập nhật trạng thái đúng/sai và lưu lịch sử bài giải vào cơ sở dữ liệu.
	3. Giao tiếp hướng sự kiện (Event Dispatcher)	Đóng gói kết quả bài làm thành sự kiện và phát bản tin bất đồng bộ lên RabbitMQ Topic Exchange.
	4. Xử lý Trò chơi hóa (Gamification)	Tiêu thụ sự kiện từ RabbitMQ để tự động cộng điểm (+10đ cho câu đúng) và kiểm tra trao huy chương ảo.
	5. Xử lý Thử thách xoay vòng (Conveyor Quests)	Tiêu thụ sự kiện bất đồng bộ để tăng chỉ số tích lũy cho Quest active, tự động trượt thử thách tiếp theo vào vị trí active khi người chơi nhận quà, và tự động reset tuần hoàn chuỗi 10 thử thách.
	6. Định tuyến và load balancing	Gateway Zuul tiếp nhận yêu cầu từ client, tra cứu danh bạ dịch vụ Eureka và điều hướng request đồng bộ đến đúng dịch vụ đích.

3.1.2. Yêu cầu phi chức năng

Để đảm bảo hệ thống microservices vận hành trơn tru trong môi trường phân tán thực tế, các yêu cầu phi chức năng sau đã được thiết lập và tối ưu hóa:

- 1. Hiệu năng và thời gian phản hồi (Performance & Latency):** Thời gian phản hồi cho các yêu cầu đồng bộ từ giao diện người dùng (nhập đáp án, xem bảng xếp hạng) phải dưới **50ms**. Các luồng hậu kỳ tốn thời gian (cộng điểm, trao huy chương, cập nhật thử thách) phải được thực hiện bất đồng bộ chạy ngầm và cam kết xử lý hoàn tất trong vòng dưới **100ms** kể từ khi sự kiện phát ra.
- 2. Khả năng chịu lỗi và tính độc lập cô lập (Resilience & Service Isolation):** Lỗi phát sinh tại dịch vụ này không được làm sập dịch vụ khác. Nếu dịch vụ Gamification hoặc Quest tạm thời ngưng hoạt động, học sinh vẫn phải thực hiện làm toán và nộp bài bình thường.

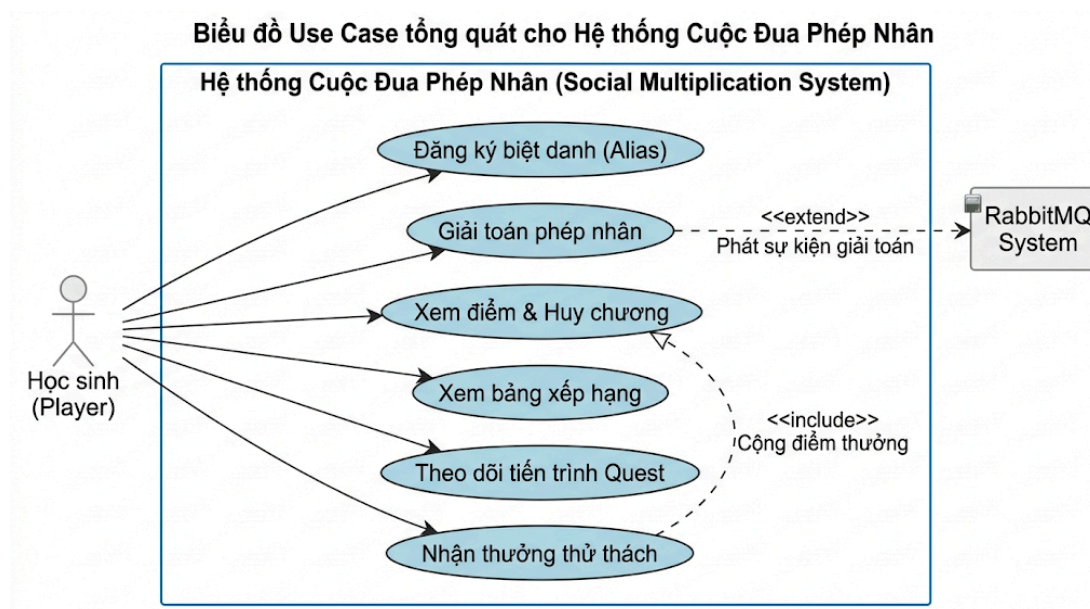
RabbitMQ có trách nhiệm bảo toàn sự kiện giải toán trong hàng đợi cho đến khi các dịch vụ kia phục hồi.

3. **Tính nhất quán dữ liệu (Data Consistency):** Hệ thống áp dụng cơ chế nhất quán cuối cùng (Eventual Consistency) thông qua các hàng đợi bất đồng bộ. Việc sử dụng cơ chế manual ACK/NACK đảm bảo tính toàn vẹn dữ liệu, loại bỏ hiện tượng thất thoát điểm số hoặc hủy chương của học sinh.
4. **Khả năng mở rộng (Scalability):** Thiết kế database cô lập (Database-per-service) cho phép từng dịch vụ (social-multiplication, gamification, quest-service) có thể scale-up hoặc scale-out độc lập tùy thuộc vào tải hệ thống mà không tạo ra điểm nghẽn tại lớp lưu trữ.

3.2. Kiến trúc tổng thể của hệ thống

3.2.1. Biểu đồ Use case tổng quát

Biểu đồ Use case tổng quát thể hiện các tương tác giữa tác nhân học sinh, hệ thống thông điệp và các phân hệ dịch vụ nghiệp vụ. Dưới đây là biểu diễn cấu trúc Use Case của hệ thống trò chơi:



Biểu đồ Use Case tổng quát cho hệ thống Cuộc Đua Phép Nhân

3.2.2. Các kịch bản chính (Use Case Specifications)

Dưới đây là đặc tả chi tiết cho 3 kịch bản sử dụng quan trọng nhất trong hệ thống:

Kịch bản 1: Học sinh thực hiện làm bài giải phép nhân

- **Tác nhân chính:** Học sinh.

- **Tiền điều kiện:** Học sinh đã đăng ký biệt danh (alias) thành công và đang ở màn hình làm toán.

- **Luồng sự kiện chính:**

1. Giao diện Web UI gửi request lấy câu hỏi ngẫu nhiên qua API Gateway.
 2. Multiplication Service tạo ngẫu nhiên phép nhân hai chữ số và gửi lại cho Web UI hiển thị.
 3. Học sinh nhập đáp án và nhấn nút gửi.
 4. Web UI gửi dữ liệu đáp án POST lên `http://localhost:8000/api/results`.
 5. Gateway Zuul chuyển tiếp request đến Multiplication Service.
 6. Multiplication Service tính toán kết quả thực tế, so khớp với đáp án của học sinh và đánh dấu trạng thái đúng/sai.
 7. Bản ghi kết quả được lưu xuống H2 Database của dịch vụ.
 8. Dịch vụ phát thông điệp MultiplicationSolvedEvent chứa chi tiết bài giải lên RabbitMQ Exchange.
 9. Hệ thống trả về response JSON chứa trạng thái đúng/sai để frontend cập nhật khung viền giao diện (đỏ/xanh neon).
- **Hậu điều kiện:** Kết quả bài làm được lưu vết vĩnh viễn, sự kiện được phát thành công lên RabbitMQ.

Kịch bản 2: Hệ thống tích lũy điểm số và trao huy chương

- **Tác nhân chính:** Hệ thống (Gamification Service).
 - **Tiền điều kiện:** Sự kiện MultiplicationSolvedEvent được đẩy thành công lên RabbitMQ.
 - **Luồng sự kiện chính:**
1. Gamification Service lắng nghe hàng đợi `gamification_multiplication_queue`, tự động nhận thông điệp sự kiện.
 2. Dịch vụ kiểm tra trạng thái đúng/sai của phép tính. Nếu sai, hệ thống bỏ qua không xử lý điểm.
 3. Nếu đúng, dịch vụ tạo bản ghi ScoreCard mới có giá trị +10 điểm gắn với ID người chơi và lưu vào database H2.
 4. Hệ thống truy vấn tổng điểm tích lũy và danh sách huy chương hiện tại của người chơi trong database.
 5. Hệ thống chạy bộ quy tắc kiểm tra trao huy chương (Badge Rules):

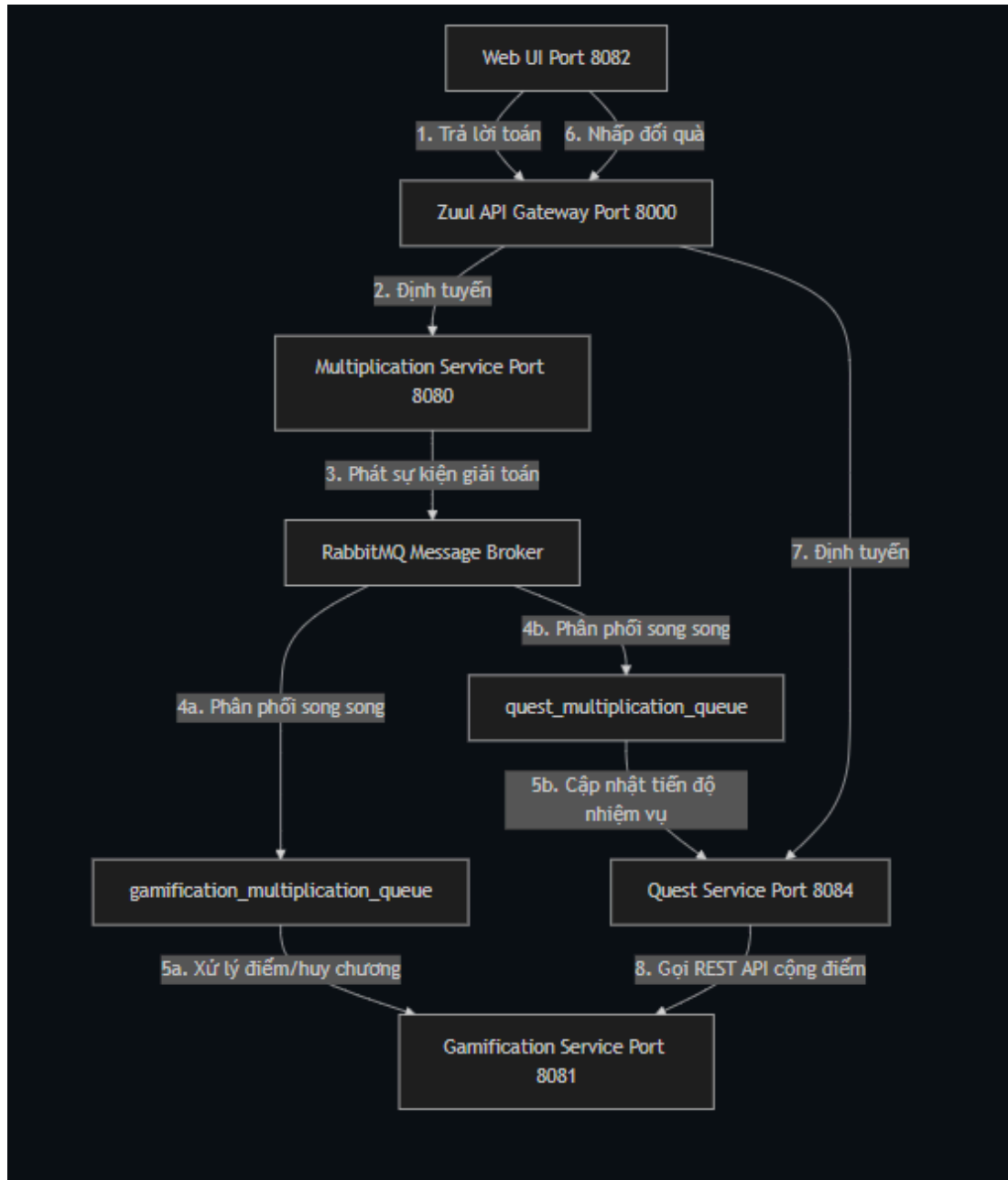
- Nếu đây là câu đúng đầu tiên -> trao huy chương FIRST_WON.
 - Nếu tổng điểm đạt mốc 100đ -> trao huy chương BRONZE_MULTIPLICATOR.
 - Nếu tổng điểm đạt mốc 500đ -> trao huy chương SILVER_MULTIPLICATOR.
 - Nếu tổng điểm đạt mốc 999đ -> trao huy chương GOLD_MULTIPLICATOR.
6. Các huy chương mới được lưu trữ dưới dạng bản ghi BadgeCard vào database.
- **Hậu điều kiện:** Điểm số và danh sách huy chương của học sinh được cập nhật chính xác và đồng bộ thời gian thực trên bảng Leaderboard.

Kịch bản 3: Hệ thống xử lý thử thách và nhận thưởng xoay vòng (Quest)

- **Tác nhân chính:** Học sinh và Hệ thống (Quest Service).
- **Tiền điều kiện:** Quest Service tiêu thụ sự kiện giải toán thành công và cập nhật thanh tiến trình đạt 100%.
- **Luồng sự kiện chính:**
 1. Trên giao diện người chơi, Quest tương ứng hiển thị nút [**Nhận +X điểm**] màu cam nổi bật.
 2. Học sinh nhấp chuột vào nút nhận thưởng.
 3. Frontend gửi POST request /api/quests/claim đến API Gateway, chuyển tiếp tới Quest Service.
 4. Quest Service xác thực tiến trình nhiệm vụ (phải completed và chưa claimed).
 5. Quest Service tạo cuộc gọi HTTP POST đồng bộ sang Gamification Service qua REST API /api/stats/award truyền DTO chứa ID học sinh và số điểm cộng thưởng.
 6. Gamification Service lưu bản ghi ScoreCard cộng điểm thưởng cho người chơi và phản hồi trạng thái thành công (HTTP 200 OK).
 7. Quest Service nhận phản hồi thành công, cập nhật cột claimed = true của bản ghi UserQuestProgress trong database của mình.
 8. Quest Service kiểm tra danh sách nhiệm vụ trong nhóm. Nếu toàn bộ đã claimed -> thực hiện reset cả 10 nhiệm vụ về mặc định để tạo vòng lặp tuần hoàn (Conveyor Board).
 9. Trả kết quả thành công về Web UI để làm mới giao diện và hiển thị thử thách tiếp theo.
- **Hậu điều kiện:** Điểm cộng thưởng được ghi nhận thành công, thử thách mới tự động trượt vào vị trí active.

3.3. Thiết kế các microservice

Kiến trúc hệ thống trò chơi giáo dục phân tán bao gồm 5 dịch vụ cốt lõi hoạt động độc lập và liên kết chặt chẽ:



Kiến trúc hệ thống

3.3.1. API Gateway

Hệ thống sử dụng **Netflix Zuul** đóng vai trò API Gateway chạy tại cổng **8000**. Mọi giao tiếp từ Web UI bên ngoài đều phải đi qua Gateway này.

- **Lợi ích:**
 - Đóng vai trò là điểm truy cập duy nhất (Single Entry Point), giấu đi kiến trúc cổng mạng phức tạp của các dịch vụ bên dưới.
 - Loại bỏ hoàn toàn lỗi cấu hình chia sẻ tài nguyên nguồn gốc chéo (CORS) ở phía client.
 - Tích hợp với Eureka để tự động cân bằng tải phía client bằng Ribbon.
- **Cấu trúc routing:** Các đường dẫn `/api/multiplications/**`, `/api/results/**` và `/api/users/**` được map về dịch vụ multiplication. Các đường dẫn `/api/leaders/**`, `/api/scores/**` và `/api/stats/**` định tuyến về dịch vụ gamification. Đường dẫn `/api/quests/**` định tuyến về dịch vụ quest.

3.3.2. Quản lý cấu hình dịch vụ (Configuration Management)

Nhằm đảm bảo tính độc lập tối đa và dễ dàng triển khai, hệ thống quản lý cấu hình phân tán trực tiếp trong tệp `application.properties` (hoặc `application.yml` đối với Gateway) riêng biệt của từng microservice.

- **Môi trường cục bộ:** Khai báo mặc định các địa chỉ kết nối như `localhost:8761` (Eureka), `localhost:5672` (RabbitMQ).
- **Tham số hóa host kết nối:** Các dịch vụ liên kết với nhau sử dụng biến môi trường có giá trị mặc định fallback (ví dụ: `${multiplicationHost:http://localhost:8000/api}`), cho phép dễ dàng ghi đè cấu hình host khi chạy thử nghiệm trên Docker Network hoặc môi trường Kubernetes thực tế mà không cần sửa đổi mã nguồn.

3.3.3. Eureka Discovery Server

Dịch vụ đăng ký và phát hiện dịch vụ **Eureka Server** chạy độc lập tại cổng **8761**.

- **Cơ chế hoạt động:** Khi khởi động, các dịch vụ `social-multiplication`, `gamification` và `quest-service` tự động gửi thông tin đăng ký (metadata, IP, Port) lên Eureka Server. Định kỳ mỗi 30 giây, các microservice gửi tín hiệu nhịp tim (Heartbeat) để xác nhận trạng thái hoạt động (“Up”).
- **Tự động định tuyến động:** API Gateway Zuul sẽ đồng bộ danh sách dịch vụ từ Eureka. Khi định tuyến, Zuul không gọi IP cứng mà dùng `serviceId` logic (ví dụ: `serviceId: multiplication`). Điều này cho phép hệ thống tự động phát hiện và định tuyến chính xác ngay cả khi các microservice thay đổi IP hoặc scale nhiều instance đồng thời.

3.3.4. Multiplication Microservice

Chịu trách nhiệm thực hiện nghiệp vụ học tập chính của trò chơi.

- **Cấu trúc gói:**
- controller: Cung cấp MultiplicationController (lấy phép tính ngẫu nhiên), MultiplicationResultAttemptController (nộp bài giải).
- domain: Chứa các thực thể JPA User, Multiplication, MultiplicationResultAttempt.
- repository: Cung cấp các repository JPA tương tác cơ sở dữ liệu.
- service: GeneratorService (sinh số ngẫu nhiên), MultiplicationService (xử lý chấm điểm bài làm).
- event: Đóng gói EventDispatcher gửi sự kiện giải toán lên RabbitMQ.

3.3.5. Gamification Microservice

Chịu trách nhiệm xử lý các bài toán nghiệp vụ liên quan đến trò chơi hóa học sinh.

- **Cấu trúc gói:**
- controller: LeaderBoardController (bảng xếp hạng Top 10), StatsController (lấy điểm/huy chương cá nhân, tiếp nhận yêu cầu cộng điểm từ Quest).
- domain: Các thực thể ScoreCard, BadgeCard và enum Badge.
- repository: ScoreCardRepository, BadgeCardRepository.
- service: GamificationService (cộng điểm tích lũy, duyệt và trao tặng huy chương dựa trên hệ thống quy tắc Badge Rules).
- event: EventHandler tiêu thụ thông điệp bất đồng bộ từ hàng đợi RabbitMQ.

3.3.6. Quest Microservice

Quản lý các chuỗi thử thách nhiệm vụ động để lôi cuốn người chơi.

- **Cấu trúc gói:**
- controller: QuestController cung cấp API lấy danh sách 3 Quest active kèm phần trăm tiến độ, và nhận yêu cầu claim thưởng của học sinh.
- domain: Thực thể Quest (30 câu hình Quest tĩnh) và UserQuestProgress (tiến độ thực nghiệm của học sinh).
- repository: QuestRepository, UserQuestProgressRepository.

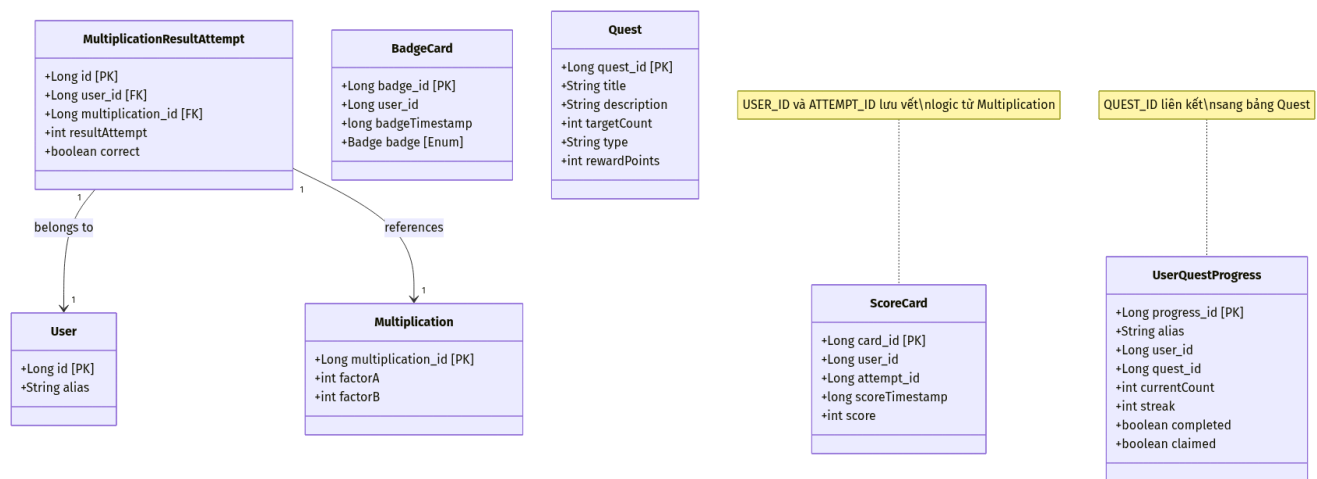
- service: QuestService xử lý logic cộng dồn tiến trình khi giải toán, kiểm tra trạng thái hoàn thành, thực hiện REST call sang Gamification để cộng thưởng, và reset xoay vòng Quest (Conveyor Board).
- event: EventHandler lắng nghe sự kiện giải toán từ RabbitMQ.

3.3.7. Giao tiếp REST liên dịch vụ (Inter-service REST call handler)

Để thực hiện giao tiếp đồng bộ giữa Quest Service và Gamification Service, một cấu hình RestClientConfiguration được thiết lập tại Quest Service để cung cấp Bean RestTemplate dùng chung. Dữ liệu trao đổi được đóng gói vào lớp DTO AwardPointsDto chứa hai trường dữ liệu: userId (ID của học sinh đạt giải) và points (số điểm thưởng của Quest tương ứng).

3.4. Thiết kế cơ sở dữ liệu

Để đảm bảo tính độc lập cao nhất, hệ thống ứng dụng mô hình **Database-per-service**. Ba cơ sở dữ liệu nhúng H2 hoạt động hoàn toàn cô lập, không chia sẻ bảng và chỉ giao tiếp thông qua các API REST hoặc RabbitMQ Events.



mermaid chart

3.4.1. Nhóm dữ liệu Multiplication Service

- **Thực thể User:**
 - USER_ID (Long, PK, Auto Generated): Khóa chính tự sinh danh tính người chơi.
 - alias (Varchar): Biệt danh học sinh nhập vào.
- **Thực thể Multiplication:**
 - MULTIPLICATION_ID (Long, PK, Auto Generated): Khóa chính của phép tính.
 - factorA (int), factorB (int): Hai số hạng nhân ngẫu nhiên.

- **Thực thể MultiplicationResultAttempt:**

- id (Long, PK, Auto Generated): Định danh bài giải.
- USER_ID (Long, FK): Liên kết đến bảng User.
- MULTIPLICATION_ID (Long, FK): Liên kết đến bảng Multiplication.
- resultAttempt (int): Đáp án học sinh nhập.
- correct (boolean): Trạng thái chấm điểm đúng/sai.

3.4.2. Nhóm dữ liệu Gamification Service

- **Thực thể ScoreCard:**

- CARD_ID (Long, PK, Auto Generated): Khóa chính định danh thẻ điểm.
- USER_ID (Long): Khóa ngoại logic trỏ sang ID của User ở Multiplication Service.
- ATTEMPT_ID (Long): Khóa ngoại logic tham chiếu đến lượt giải toán đúng tương ứng.
- scoreTimestamp (bigint): Thời điểm ghi nhận điểm cộng.
- score (int): Số điểm được cộng (mặc định là +10đ cho câu đúng, hoặc +Xđ cho điểm thưởng Quest).

- **Thực thể BadgeCard:**

- BADGE_ID (Long, PK, Auto Generated): Định danh huy chương được cấp.
- USER_ID (Long): Khóa ngoại logic người dùng.
- badgeTimestamp (bigint): Thời điểm nhận huy chương.
- badge (Varchar/Enum): Loại huy chương được cấp (BRONZE_MULTIPLICATOR, SILVER_MULTIPLICATOR, GOLD_MULTIPLICATOR, FIRST_ATTEMPT, FIRST_WON, LUCKY_NUMBER).

3.4.3. Nhóm dữ liệu Quest Service

- **Thực thể Quest:**

- QUEST_ID (Long, PK, Auto Generated): Định danh Quest.
- title (Varchar), description (Varchar): Tiêu đề và nội dung mô tả của thử thách.
- targetCount (int): Chỉ số yêu cầu hoàn thành (ví dụ: đúng 3 câu, đạt streak 5 câu...).
- type (Varchar): Loại thử thách (TOTAL_CORRECT, STREAK, TOTAL_ATTEMPTS).
- rewardPoints (int): Giá trị điểm thưởng khi hoàn thành (+20, +50, +100).

- **Thực thể UserQuestProgress:**

- PROGRESS_ID (Long, PK, Auto Generated): Khóa chính định danh tiến trình.
- alias (Varchar): Biệt danh học sinh.
- user_id (Long): Lưu ID người dùng từ Multiplication Service để đồng bộ.
- questId (Long): Khóa ngoại logic liên kết đến thực thể Quest.
- currentCount (int): Chỉ số hiện tại học sinh đạt được.
- streak (int): Chuỗi trả lời đúng hiện tại của học sinh.
- completed (boolean): Đánh dấu đã hoàn thành thử thách (100%).
- claimed (boolean): Đánh dấu học sinh đã bấm nhận quà thành công.

3.5. Thiết kế RESTful API

Hệ thống cung cấp hệ thống RESTful API rõ ràng, phân vùng định tuyến tối ưu qua Zuul Gateway:

3.5.1. API cho Multiplication Service

Cung cấp các endpoints cốt lõi phục vụ bài học của học sinh:

Bảng 9. Các API của dịch vụ Multiplication

HTTP Method	Endpoint Path	Tham số yêu cầu	Kiểu dữ liệu trả về	Mô tả chức năng
GET	/api/multiplications/random	Không	JSON Object Multiplication	Sinh ngẫu nhiên câu hỏi phép nhân mới chứa hai thừa số factorA, factorB.
POST	/api/results	JSON Body MultiplicationResultAttempt	JSON Object Chấm điểm	Nhận bài giải của học sinh, tiến hành chấm điểm đúng/sai, lưu cơ sở dữ liệu và phát sự kiện lên RabbitMQ.
GET	/api/results	Query: alias={alias}	JSON Array Attempts	Lấy danh sách 5 lượt làm bài gần đây nhất của một học sinh cụ thể để hiển thị lịch sử làm bài.

GET	/api/users/{userId}	Path: userId (Long)	JSON Object User	Truy vấn thông tin tài khoản (biệt danh) dựa trên ID để phục vụ các dịch vụ liên kết đồng bộ thông tin.
------------	---------------------	---------------------	---------------------	---

3.5.2. API cho Gamification Service

Phục vụ việc hiển thị bảng thành tích và tương tác cộng điểm thưởng:

Bảng 10. Các API của dịch vụ Gamification

HTTP Method	Endpoint Path	Tham số yêu cầu	Kiểu dữ liệu trả về	Mô tả chức năng
GET	/api/leaders	Không	JSON Array LeaderBoardRow	Truy vấn danh sách Top 10 học sinh xuất sắc nhất hệ thống sắp xếp theo điểm giảm dần.
GET	/api/stats	Query: alias={alias}	JSON Object GameStats	Truy xuất tổng điểm tích lũy và danh sách huy chương hiện tại của một học sinh để hiển thị bảng điểm.
POST	/api/stats/award	JSON Body AwardPointsDto (userId, points)	HTTP Status 200 OK	Tiếp nhận yêu cầu cộng điểm thưởng từ Quest Service, tạo ScoreCard cộng điểm trực tiếp cho học sinh.

3.5.3. API cho Quest Service

Quản lý các tương tác liên quan đến thử thách nhiệm vụ:

Bảng 11. Các API của dịch vụ Quest

HTTP Method	Endpoint Path	Tham số yêu cầu	Kiểu dữ liệu trả về	Mô tả chức năng
-------------	---------------	-----------------	---------------------	-----------------

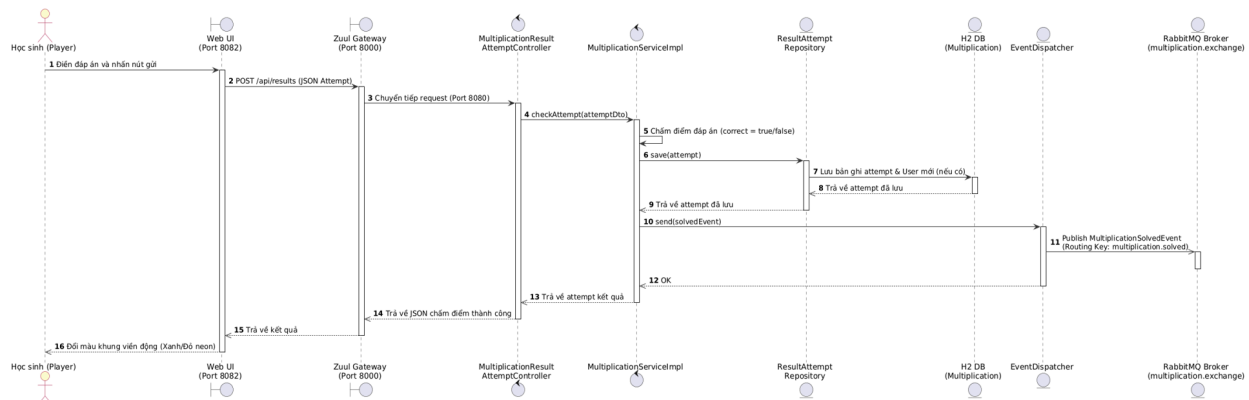
GET	/api/quests	Query: alias={alias}	JSON Array QuestProgressDto	Trả về thông tin chi tiết của 3 thử thách đang active ứng với tài khoản học sinh kèm phần trăm tiến trình hiện tại.
POST	/api/quests/claim	Query: alias={alias}, questId={id}	JSON Object ClaimResultDto	Học sinh nộp yêu cầu nhận thưởng điểm của Quest đã xong, thực hiện REST call sang Gamification để cộng thưởng.

3.6. Thiết kế luồng Message Queue và Biểu đồ tuần tự (Sequence Diagram)

Luồng tương tác phân tán hướng sự kiện và gọi REST đồng bộ giữa các dịch vụ được đặc tả bằng các biểu đồ tuần tự chi tiết dưới đây:

3.6.1. Luồng Giải toán và Xuất bản Sự kiện (Event Publishing)

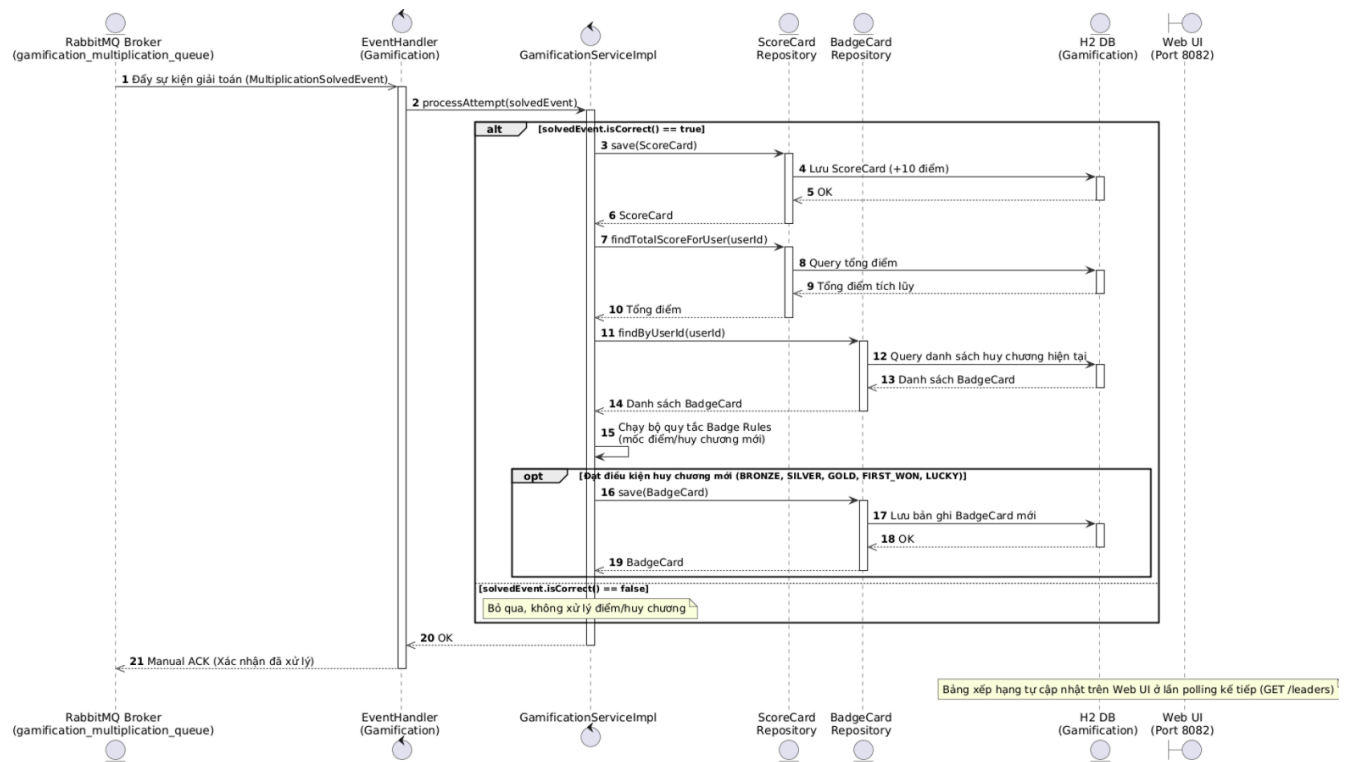
Biểu đồ tuần tự dưới đây biểu diễn quy trình từ lúc Học sinh nộp đáp án, Multiplication Service thực hiện chấm điểm, lưu database, phát sự kiện lên RabbitMQ và trả kết quả về giao diện người chơi:



mermaid chart

3.6.2. Luồng Xử lý Điểm số và Huy chương Bất đồng bộ (Gamification)

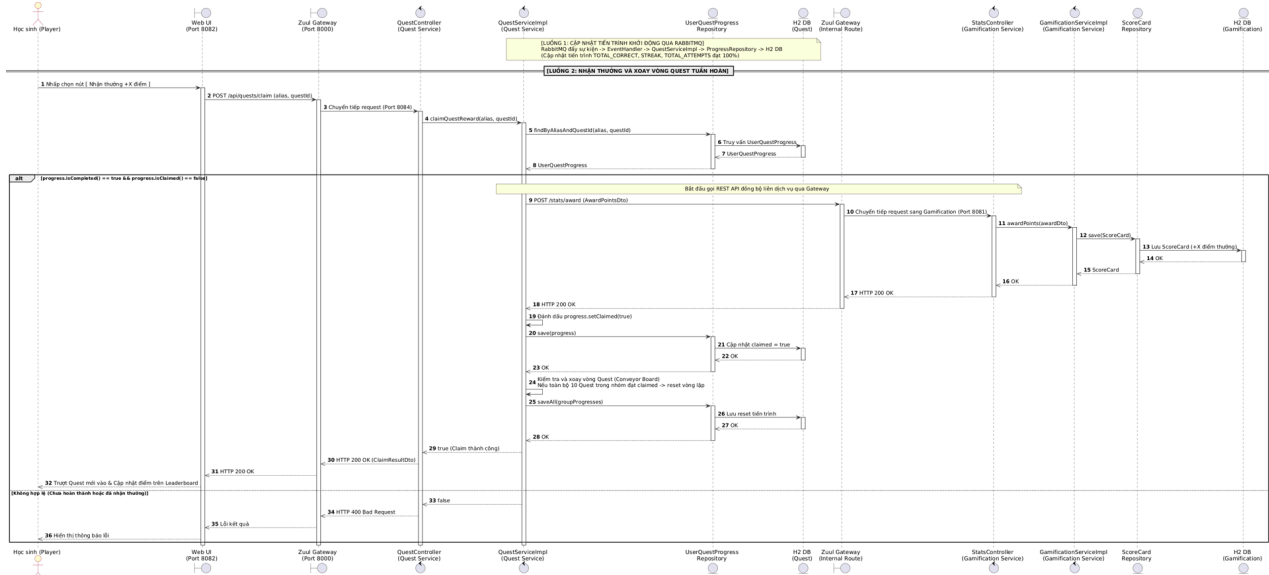
Mô tả luồng Gamification Service nhận sự kiện bất đồng bộ từ hàng đợi RabbitMQ, cộng điểm tích lũy học tập và chạy bộ quy tắc kiểm tra trao huy chương danh dự cho học sinh:



mermaid chart

3.6.3. Luồng Cập nhật Tiến trình Quest và Nhận thưởng (Quest Progress & Claim Reward)

Mô tả quy trình Quest Service nhận sự kiện giải toán qua RabbitMQ để tăng tiến độ. Tiếp theo là luồng Học sinh nhấn nút “Nhận thưởng”, Quest Service tạo cuộc gọi REST đồng bộ qua Gateway sang Gamification Service để cộng điểm và xoay vòng thử thách tiếp theo:



mermaid chart

3.7. Thiết kế bảo mật hệ thống

Nhằm đảm bảo hoạt động an toàn trong môi trường mạng nội bộ trường học, hệ thống trò chơi giáo dục “Cuộc Đua Phép Nhân” thiết lập các hàng rào bảo mật nhiều lớp:

- 1. Bảo vệ ranh giới mạng (Network Boundary):** Các cổng dịch vụ nội bộ của social-multiplication (8080), gamification (8081) và quest-service (8084) chỉ được cấu hình để lắng nghe các kết nối cục bộ nội bộ (local network interfaces) hoặc cấu hình tường lửa chỉ chấp nhận các kết nối đến từ địa chỉ IP của API Gateway Zuul. Học sinh từ bên ngoài mạng không thể quét cổng hoặc gọi trực tiếp đến các cổng dịch vụ nghiệp vụ này.
- 2. Định tuyến tập trung và ẩn giấu hạ tầng (Gateway Hiding):** API Gateway Zuul là điểm tiếp nhận duy nhất. Zuul ẩn giấu hoàn toàn địa chỉ IP thật của cơ sở dữ liệu và các microservices nghiệp vụ. Mọi truy cập trải phép cố tình quét thông tin các cổng con đều bị ngăn chặn.
- 3. Cô lập và bảo mật Cơ sở dữ liệu nhúng H2:** Các tệp cơ sở dữ liệu nhúng H2 (~social-multiplication, ~/gamification, ~/quest) được bảo vệ bằng cơ chế mã hóa quyền truy cập của hệ điều hành. Chỉ tiến trình chạy Java ảo JVM của microservice tương ứng mới có đặc quyền đọc ghi vào file này, loại bỏ rủi ro bị tấn công chèn ép (SQL Injection) từ bên ngoài.
- 4. Định cấu hình CORS nghiêm ngặt:** API Gateway Zuul được thiết lập bộ lọc CORS chỉ cho phép các yêu cầu AJAX gửi đến từ nguồn gốc Web UI hợp lệ (http://localhost:8082), ngăn chặn các hành vi giả mạo yêu cầu chéo trang độc hại (CSRF - Cross-Site Request Forgery).

3.8. Kết chương

Chương 3 đã phân tích toàn diện yêu cầu chức năng và phi chức năng của hệ thống trò chơi giáo dục “**Cuộc Đua Phép Nhân**”. Báo cáo đã đặc tả chi tiết kiến trúc Use Case hệ thống, thiết kế ranh giới các microservice, sơ đồ cơ sở dữ liệu độc lập H2 nhúng dạng tệp bền vững và hệ thống API RESTful phân khu định tuyến qua Zuul Gateway. Đồng thời, các biểu đồ tuần tự Sequence chuẩn chỉnh đã làm rõ cách thức vận hành mượt mà của luồng sự kiện bất đồng bộ qua RabbitMQ Topic Exchange và luồng gọi REST nội bộ đồng bộ khi trao quà thử thách. Đây là nền tảng thiết kế kiến trúc phân tán vững chắc để tiến hành xây dựng và triển khai ứng dụng thực nghiệm chi tiết ở chương tiếp theo.

CHƯƠNG 4: XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG “CUỘC ĐUA PHÉP NHÂN”

4.1. Môi trường phát triển và công nghệ sử dụng

Hệ thống trò chơi giáo dục toán học “**Cuộc Đua Phép Nhân**” (**Social Multiplication & Gamification**) được xây dựng và phát triển dựa trên kiến trúc microservice phân tán tiên tiến. Hệ thống sử dụng ngôn ngữ lập trình **Java 17** kết hợp với framework **Spring Boot** ở phía backend, trong khi giao diện người dùng (frontend) được thiết kế theo phong cách **Brutalism** trực quan và sinh động bằng công nghệ **HTML/JS/CSS** thuần túy. Hạ tầng trao đổi thông điệp bất đồng bộ hướng sự kiện (Event-Driven Architecture) được vận hành qua **RabbitMQ**, và toàn bộ dịch vụ đăng ký, phát hiện dịch vụ và định tuyến được quản trị bởi **Netflix Eureka** và **Zuul API Gateway**.

Dưới đây là bảng tổng hợp chi tiết về môi trường phát triển và hệ sinh thái công nghệ được áp dụng trong toàn bộ dự án:

Bảng 14. Công nghệ sử dụng trong hệ thống “Cuộc Đua Phép Nhân”

Nhóm công nghệ	Công nghệ sử dụng	Vai trò trong hệ thống
Ngôn ngữ nền	Java 17, HTML5, JavaScript (ES6), CSS3	Phát triển logic nghiệp vụ và giao diện người dùng
Backend Framework	Spring Boot 2.x, Spring Cloud	Khung phát triển các dịch vụ Microservices độc lập

API Gateway	Netflix Zuul (Cổng 8000)	Cổng tiếp nhận duy nhất, định tuyến (Routing) và load balancing
Service Discovery	Netflix Eureka Server (Cổng 8761)	Đăng ký dịch vụ động và phát hiện dịch vụ tự động
Cơ sở dữ liệu	H2 Database (File-based Embedded)	Lưu trữ dữ liệu nghiệp vụ riêng cho từng dịch vụ
Message Broker	RabbitMQ (Cổng 5672 / 15672)	Truyền tải thông điệp bất đồng bộ hướng sự kiện
Kiểm thử tự động	Cucumber, Gherkin, JUnit	Thiết lập bộ kịch bản kiểm thử E2E tích hợp tự động
Web UI Deploy	Python 3 HTTP Server (Cổng 8082)	Khởi tạo máy chủ web tĩnh phục vụ giao diện người chơi
Hạ tầng cục bộ	Docker, Docker Compose	Khởi chạy RabbitMQ container nhanh chóng và cô lập

4.1.1. Backend Microservices

Backend của hệ thống được chia nhỏ thành 5 dịch vụ thành phần độc lập hoạt động trong mạng lưới hướng cấu hình của Spring Cloud:

- **Service Registry (service-registry):** Eureka Server đóng vai trò là danh bạ điện thoại toàn hệ thống. Mọi instance của các dịch vụ nghiệp vụ khi khởi động sẽ tự động đăng ký địa chỉ IP và số hiệu cổng (port) hoạt động tại đây.
- **Zuul API Gateway (gateway):** Đóng vai trò là “người gác cổng” duy nhất cho hệ thống backend. Client (Web UI) chỉ tương tác thông qua cổng 8000 của Gateway. Zuul sẽ tự động tra cứu Eureka và định tuyến các request đến dịch vụ thích hợp dưới tiền tố /api.
- **Multiplication Service (social-multiplication):** Xử lý nghiệp vụ cốt lõi về người dùng (chơi với tư cách ẩn danh qua biệt danh - alias), tự động phát sinh câu hỏi phép nhân ngẫu nhiên, tiếp nhận bài làm của học sinh, chấm điểm và lưu lịch sử làm bài. Đồng thời dịch vụ này phát ra các sự kiện khi câu hỏi được giải quyết thành công/thất bại lên RabbitMQ.

- **Gamification Service (gamification):** Xử lý nghiệp vụ trò chơi hóa (gamification). Dịch vụ này lắng nghe sự kiện từ RabbitMQ để cộng điểm tích lũy, cấp huy chương (Badges) tương ứng và quản lý bảng xếp hạng (Leaderboard) thời gian thực.
- **Quest Service (quest-service):** Dịch vụ quản lý các thử thách xoay vòng tuần hoàn (30 Quest chia đều cho 3 cấp độ: Dễ, Trung bình, Khó). Dịch vụ này hoạt động bất đồng bộ bằng cách tiêu thụ sự kiện giải toán từ RabbitMQ để cập nhật tiến trình của 3 Quest đang hiển thị trên giao diện người dùng. Khi người dùng bấm nhận thưởng, Quest Service sẽ tạo một REST gọi đồng bộ sang Gamification Service để cộng điểm thưởng tương ứng.

4.1.2. Frontend Web UI

Giao diện người dùng được thiết kế bằng **HTML5, CSS3 và Vanilla JavaScript** thuần túy không phụ thuộc vào các framework nặng nề như React hay Angular nhằm tối ưu tốc độ tải và khả năng hiển thị trực tiếp trên trình duyệt. Phong cách thiết kế được áp dụng là **Brutalism** - một xu hướng thiết kế hiện đại sử dụng các tông màu tương phản mạnh (xanh Neon, vàng chanh, hồng cánh sen), các đường viền đen đậm dày và phông chữ Serif lớn, tạo cảm giác vô cùng phá cách, năng động và kích thích sự tò mò của học sinh tiểu học khi học toán. Giao diện tích hợp kỹ thuật bất đồng bộ **Fetch API** để gửi kết quả giải toán, truy vấn bảng xếp hạng và cập nhật tiến trình thử thách liên tục mà không cần tải lại toàn bộ trang (Zero Reload).

4.1.3. Cơ sở dữ liệu và hạ tầng thông điệp

- **Cơ sở dữ liệu H2 Database:** Nhằm đơn giản hóa quá trình cài đặt và kiểm thử nội bộ nhưng vẫn đảm bảo tính độc lập dữ liệu giữa các microservices, mỗi dịch vụ (social-multiplication, gamification, quest-service) đều tích hợp một cơ sở dữ liệu nhúng H2 dạng tệp tin (file-based). Dữ liệu được ghi trực tiếp xuống các file cục bộ trong thư mục người dùng (~/.social-multiplication, ~/.gamification, ~/.quest), giúp bảo toàn dữ liệu sau mỗi lần khởi động lại dịch vụ.
- **Hàng đợi RabbitMQ:** Hoạt động như mạch máu liên kết các dịch vụ theo cơ chế bất đồng bộ. RabbitMQ được triển khai cô lập thông qua Docker Container giúp tối ưu hiệu năng truyền tải tin nhắn dạng JSON, cam kết tính nhất quán dữ liệu giữa luồng học tập (Multiplication) và luồng giải thưởng (Gamification & Quest).

4.1.4. Công cụ kiểm thử và vận hành

- **Docker Compose:** Được sử dụng để định nghĩa và khởi chạy nhanh chóng RabbitMQ Server cùng bảng quản trị Web (Management Plugin) chỉ với một câu lệnh đơn giản, loại bỏ hoàn toàn rào cản cài đặt môi trường phức tạp trên hệ điều hành của người dùng.
- **Maven Wrapper (mvnw):** Đảm bảo tính nhất quán về phiên bản build Maven trên mọi máy tính cục bộ của các lập trình viên/giáo viên kiểm tra mà không cần cài đặt sẵn Maven toàn cục.
- **Python 3 HTTP Server:** Công cụ nhanh chóng để khởi tạo một Web Server tĩnh cục bộ chạy ở cổng 8082, phục vụ giao diện người chơi một cách nhẹ nhàng và ổn định nhất.

- **Cucumber & Gherkin:** Tích hợp trong thư mục tests_e2e giúp tự động hóa quá trình kiểm thử các kịch bản sử dụng thực tế (End-to-End Test) bằng ngôn ngữ tự nhiên gần gũi với con người, giúp đảm bảo toàn bộ hệ thống phân tán hoạt động hoàn hảo trước khi phát hành.

4.2. Xây dựng Backend Microservices

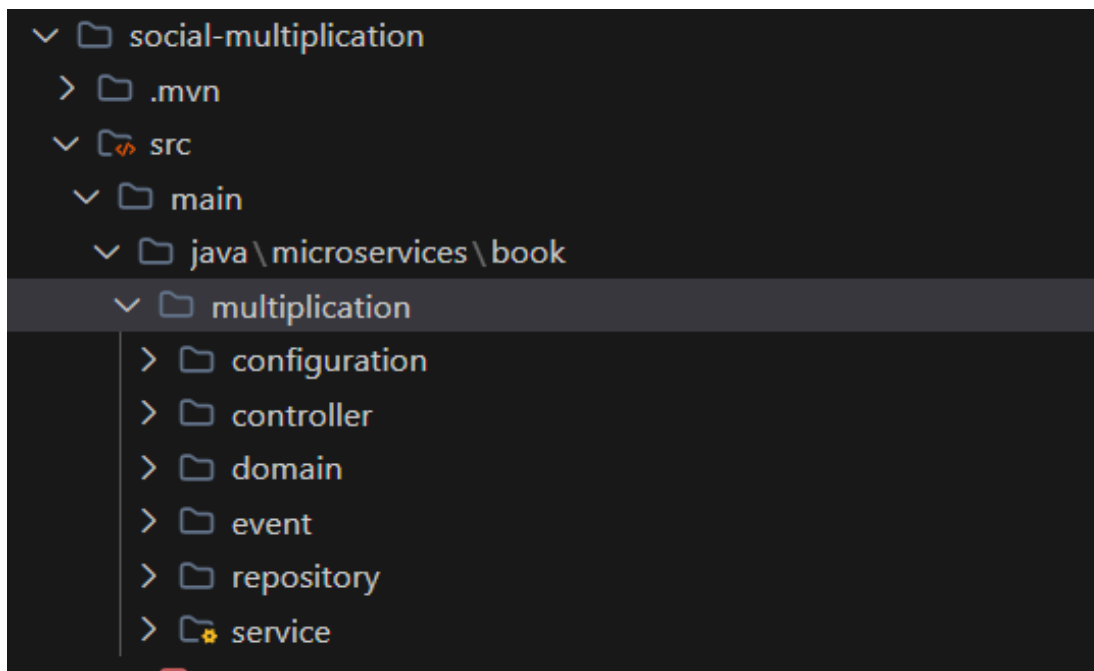
Kiến trúc backend của hệ thống trò chơi “Cuộc Đua Phép Nhân” tuân thủ nghiêm ngặt nguyên lý thiết kế microservices, trong đó mỗi dịch vụ được đóng gói hoàn chỉnh từ tầng Controller, Service, Repository cho tới cấu hình kết nối mạng.

```
c:\CHTPT
├── service-registry/      # Eureka Server (Port 8761)
├── gateway/              # Zuul API Gateway (Port 8000)
├── social-multiplication/ # Multiplication Microservice (Port 8080)
├── gamification/         # Gamification Microservice (Port 8081)
└── quest-service/        # Quest Microservice (Port 8084)
```

4.2.1. Xây dựng Multiplication Service (social-multiplication)

Dịch vụ này quản lý logic cốt lõi liên quan đến các phép tính và lưu trữ thông tin thực nghiệm của người dùng.

- **Phát sinh phép tính ngẫu nhiên:** Sử dụng GeneratorService để sinh ngẫu nhiên hai thừa số trong khoảng từ 11 đến 99, đảm bảo thử thách độ khó vừa phải cho học sinh thực hành phép nhân hai chữ số.
- **Xác thực và lưu trữ kết quả:** Lớp MultiplicationServiceImpl tiếp nhận biệt danh (alias) và đáp án của người dùng. Nếu người dùng chưa tồn tại trong cơ sở dữ liệu, dịch vụ sẽ tự động tạo mới bản ghi User. Kết quả bài làm được đóng gói vào đối tượng MultiplicationResultAttempt và lưu xuống database H2 thông qua ResultAttemptRepository.
- **Phát hành sự kiện:** Khi một bài làm được xử lý, dịch vụ sẽ lập tức phát đi một thông điệp MultiplicationSolvedEvent lên RabbitMQ thông qua đối tượng EventDispatcher. Event này chứa thông tin định danh bài giải, ID người dùng, trạng thái đúng/sai và giá trị của hai thừa số nhằm phục vụ cho các dịch vụ Gamification và Quest tiêu thụ phía sau.



Sơ đồ cấu trúc thư mục của dịch vụ social-multiplication

4.2.2. Xây dựng Gamification Service (gamification)

Dịch vụ này chịu trách nhiệm tích lũy điểm số, xếp hạng và trao tặng các danh hiệu ảo cho người chơi nhằm duy trì động lực học tập.

- **Tiêu thụ sự kiện từ RabbitMQ:** Lớp EventHandler lắng nghe hàng đợi gamification_multiplication_queue. Khi nhận được MultiplicationSolvedEvent, nó sẽ gọi dịch vụ GamificationServiceImpl để xử lý.
- **Tích lũy điểm và cấp huy chương:** Mỗi câu trả lời đúng được cộng 10 điểm. Hệ thống sẽ tra cứu tổng số điểm hiện tại và lịch sử huy chương của người dùng thông qua các bảng ScoreCard và BadgeCard. Tùy thuộc vào các mốc điểm và điều kiện đạt được, dịch vụ sẽ trao tặng các huy chương tương ứng được định nghĩa trong enum Badge:
- **Huy chương theo điểm số:** BRONZE_MULTIPLICATOR (sau khi đạt 100 điểm), SILVER_MULTIPLICATOR (đạt 500 điểm), và GOLD_MULTIPLICATOR (đạt 999 điểm).
- **Huy chương theo sự kiện đặc biệt:** FIRST_ATTEMPT (cho lượt giải toán đầu tiên, bất kể đúng hay sai), FIRST_WON (cho câu trả lời đúng đầu tiên), và LUCKY_NUMBER (nếu giải đúng phép tính có sự xuất hiện của các con số may mắn đặc trưng).
- **Cung cấp API Leaderboard:** LeaderBoardController cung cấp các API để truy cập bảng xếp hạng Top 10 người chơi có điểm số cao nhất hệ thống, giúp kích thích tính ganh đua lành mạnh giữa các học sinh.

- **API cộng điểm thưởng đặc biệt:** Cung cấp endpoint POST /stats/award tiếp nhận đối tượng AwardPointsDto để cho phép dịch vụ ngoại vi (cụ thể là quest-service) cộng điểm thưởng trực tiếp cho người chơi khi họ hoàn thành các nhiệm vụ khó.

4.2.3. Xây dựng Quest Service (quest-service)

Đây là dịch vụ mới được phát triển nhằm tăng tính hấp dẫn bằng các chuỗi thử thách năng động.

- **Hệ thống 30 thử thách xoay vòng:** Trong phương thức @PostConstruct initQuests(), dịch vụ tự động khởi tạo 30 Quest độc đáo được chia làm 3 cấp độ điểm thưởng tương ứng:
- **Cấp Dễ (+20 điểm - Tân Binh Nhảm Thuật):** Gồm 10 Quest như “*Khởi đầu nhẹ nhàng*” (giải đúng 3 phép tính), “*Chuỗi khởi động*” (chuỗi 3 câu đúng liên tiếp), “*Chăm chỉ học tập*” (làm 5 lượt bài),...
- **Cấp Trung Bình (+50 điểm - Chiến Thần Tốc Độ):** Gồm 10 Quest có yêu cầu cao hơn như “*Bản lĩnh trung cấp*” (đúng 8 câu), “*Chuỗi bút phá*” (chuỗi 6 câu đúng),...
- **Cấp Khó (+100 điểm - Huyền Thoại Toán Học):** Gồm 10 Quest đòi hỏi sự kiên trì và tư duy vượt bậc như “*Đỉnh cao thử thách*” (đúng 18 câu), “*Chuỗi siêu đẳng*” (chuỗi 10 câu đúng liên tiếp), “*Vua phép nhân*” (đúng 25 câu)...
- **Cập nhật tiến trình động:** Lớp EventHandler của dịch vụ tiêu thụ sự kiện từ hàng đợi quest_multiplication_queue. Nó sẽ tìm kiếm các Quest đang **active** (đang hiển thị cho người chơi) và cập nhật tiến trình dựa vào loại chỉ số: TOTAL_CORRECT (cộng dồn số câu đúng), STREAK (tăng chuỗi đúng liên tiếp hoặc reset về 0 nếu trả lời sai), và TOTAL_ATTEMPTS (tăng số lượt thử).
- **Cơ chế gọi REST liên dịch vụ:** Khi người dùng hoàn thành 100% tiến trình của một Quest và nhấp nút “*Nhận thưởng*”, QuestServiceImpl sẽ gửi một yêu cầu HTTP POST đồng bộ sang Gamification Service thông qua RestTemplate để cộng điểm thưởng tương ứng (+20, +50 hoặc +100 điểm). Khi Gamification xác nhận thành công, trạng thái Quest đó sẽ được cập nhật là claimed = true trong H2 database của Quest Service.

4.2.4. Xây dựng Zuul API Gateway (gateway)

Zuul Gateway đóng vai trò làm trung gian phân phối request cho toàn bộ hệ thống microservices. Cấu hình định tuyến chi tiết được lưu trữ trong tệp application.yml như sau:

yaml

server:

port: 8000

zuul:

ignoredServices: '*'

prefix: /api

routes:

 multiplications:

 path: /multiplications/**

 serviceId: multiplication

 strip-prefix: false

 results:

 path: /results/**

 serviceId: multiplication

 strip-prefix: false

 users:

 path: /users/**

 serviceId: multiplication

 strip-prefix: false

 leaders:

 path: /leaders/**

 serviceId: gamification

 strip-prefix: false

 scores:

 path: /scores/**

 serviceId: gamification

 strip-prefix: false

 stats:

 path: /stats/**

 serviceId: gamification

 strip-prefix: false

 quests:

```
path: /quests/**  
serviceId: quest  
strip-prefix: false
```

Nhờ cấu hình này, toàn bộ các yêu cầu từ Web UI sẽ được quy về một địa chỉ IP duy nhất là `http://localhost:8000/api/...`, giúp loại bỏ hoàn toàn các rắc rối liên quan đến cấu hình chia sẻ tài nguyên giữa các nguồn khác nhau (CORS - Cross-Origin Resource Sharing) ở phía Client.

4.2.5. Xây dựng Eureka Service Discovery Server (service-registry)

Dịch vụ này được kích hoạt thông qua annotation `@EnableEurekaServer` đặt tại lớp khởi chạy `ServiceRegistryApplication`. Eureka Server chạy tại cổng mặc định 8761. Mọi microservice nghiệp vụ đều khai báo cấu hình client để đăng ký thông tin của mình lên Eureka Server:

properties

```
eureka.client.service-url.default-zone=http://localhost:8761/eureka/
```

Khi API Gateway Zuul tiếp nhận một request, nó sẽ dựa vào `serviceId` đã cấu hình để truy vấn Eureka Server, lấy ra danh sách các IP và Port đang hoạt động của dịch vụ đó, sau đó định tuyến request một cách chính xác và cân bằng tải hiệu quả.

4.3. Xây dựng Frontend UI

Giao diện người chơi (Web UI) được xây dựng theo phong cách **Brutalism** hiện đại, mang tính đột phá cao về mặt thẩm mỹ kỹ thuật số nhằm lôi cuốn học sinh tiểu học. Mã nguồn tĩnh của giao diện nằm hoàn toàn trong thư mục `ui/webapps/ui/` bao gồm tệp cấu trúc `index.html`, tệp định dạng thẩm mỹ `style.css` và tệp điều khiển logic `multiplication-client.js`.

4.3.1. Giao diện trang chủ và làm toán

Khi người chơi truy cập vào ứng dụng, một màn hình đăng nhập tối giản và ấn tượng sẽ xuất hiện yêu cầu người chơi điền biệt danh (alias) của mình. Biệt danh này đóng vai trò là danh tính xuyên suốt trong game để lưu trữ bảng điểm và thứ hạng.

Giao diện màn hình đăng nhập biệt danh của trò chơi

Sau khi nhập biệt danh và nhấn nút gửi, giao diện trò chơi chính thức sẽ trượt ra vô cùng mượt mà. Hệ thống hiển thị câu hỏi phép nhân ngẫu nhiên dạng bảng phần lớn. Học sinh nhập đáp án và nhấn nút gửi. Kết quả đúng/sai sẽ được hiển thị ngay lập tức kèm theo hiệu ứng đổi màu khung viền động (Xanh lá nếu đúng, Đỏ neon nếu sai) tạo phản hồi thị giác mạnh mẽ.

Giao diện làm bài phép tính nhân thời gian thực

4.3.2. Giao diện Bảng xếp hạng và Bảng huy chương

Phía bên phải giao diện chính là khu vực dành riêng cho việc vinh danh thành tích học tập:

- **Bảng huy chương (Badges):** Hiển thị danh sách các huy chương mà người chơi hiện tại đang sở hữu dưới dạng các icon tròn có viền đen dày dặn theo phong cách Brutalism. Mỗi khi người chơi đạt đủ điểm số hoặc đạt mốc đầu tiên, huy chương tương ứng sẽ sáng lên một cách đầy tự hào.



Giao diện huy chương danh dự của người chơi

- **Bảng xếp hạng (Leaderboard):** Danh sách Top 10 học sinh có điểm số cao nhất toàn trường được cập nhật tự động sau mỗi giây thông qua cơ chế gửi Ajax request định kỳ lên dịch vụ Gamification.

 BẢNG XẾP HẠNG TẢI LẠI 		
HẠNG	NGƯỜI CHƠI	TỔNG ĐIỂM
01	Tran_tu	350 điểm
02	Hoang	100 điểm
03	tran_tu	20 điểm
4	hoang	10 điểm

Giao diện Bảng xếp hạng

4.3.3. Giao diện Thử thách nhiệm vụ (Quests Dashboard)

Khung giao diện “**Thử Thách Nhận Thưởng (Quests)**” là một đột phá về trải nghiệm người dùng trong dự án. Khung này được bố trí ngay dưới khu vực làm bài, hiển thị song song **3 thẻ nhiệm vụ active** tương ứng với 3 cấp độ điểm thưởng:

- **Thẻ màu Xanh Neon (Dễ - +20đ):** Hiển thị tên thử thách, thanh tiến trình dạng phần trăm và số lượng bài đã hoàn thành (ví dụ: 2/3).
- **Thẻ màu Vàng Chanh (Trung bình - +50đ):** Hiển thị thử thách cấp trung.
- **Thẻ màu Hồng Cam (Khó - +100đ):** Thử thách cấp khó nhất dành cho những học sinh xuất sắc.

THỬ THÁCH NHẬN THƯỞNG

KHỞI ĐẦU NHẸ NHÀNG +20 ĐIỂM
Giải đúng 3 phép tính phép nhân bất kỳ

1/3

BẢN LĨNH TRUNG CẤP +50 ĐIỂM
Giải đúng 8 phép tính phép nhân bất kỳ

1/8

ĐỈNH CAO THỬ THÁCH +100 ĐIỂM
Giải đúng 18 phép tính phép nhân bất kỳ

1/18

Giao diện khung Thử thách Quests động

4.4. Tích hợp giao tiếp đồng bộ RESTful API qua API Gateway

Giao tiếp đồng bộ được áp dụng cho toàn bộ các luồng xử lý yêu cầu phản hồi lập tức từ phía Client đến Backend và định tuyến qua Zuul API Gateway.

- Khi người chơi gửi một đáp án, frontend gửi một yêu cầu POST đồng bộ đến `http://localhost:8000/api/results`. Gateway Zuul tiếp nhận, tra cứu Eureka và chuyển tiếp yêu cầu đến Multiplication Service tại cổng 8080. Dịch vụ này lập tức chấm điểm và trả về kết quả JSON dạng: `{ "correct": true, "user": { "id": 1, "alias": "thanh_cong" }, "valA": 45, "valB": 12, "attempt": 540 }`
- Tương tự, bảng xếp hạng được truy vấn thông qua phương thức GET gửi đến `http://localhost:8000/api/leaders`, Zuul định tuyến yêu cầu đến cổng 8081 của Gamification Service để trả về danh sách xếp hạng dạng mảng JSON chỉ trong vài mili-giây.

4.5. Cơ chế gọi REST liên dịch vụ (Inter-service Communication)

Bên cạnh giao tiếp bất đồng bộ bằng sự kiện, hệ thống sử dụng giao tiếp đồng bộ bằng REST API giữa các dịch vụ nội bộ khi có yêu cầu cam kết giao dịch tức thời. Cụ thể là luồng **Nhận thưởng nhiệm vụ** giữa Quest Service và Gamification Service:

1. Khi học sinh hoàn thành một Quest, trên giao diện sẽ xuất hiện nút [**Nhận +X điểm**] màu cam nổi bật.
2. Khi học sinh nhấp nút này, frontend gửi một POST request đến Gateway: POST /api/quests/claim kèm tham số alias và questId.
3. Gateway chuyển tiếp yêu cầu đến Quest Service.
4. Lớp QuestServiceImpl nhận yêu cầu, tiến hành kiểm tra tính hợp lệ của Quest (phải hoàn thành và chưa được nhận thưởng). Sau đó, nó sử dụng thư viện RestTemplate để thực hiện một cuộc gọi HTTP POST đồng bộ sang Gamification Service: String url = gamificationHost + "/stats/award"; AwardPointsDto awardDto = new AwardPointsDto(progress.getUserId(), quest.getRewardPoints()); restTemplate.postForObject(url, awardDto, Object.class);
5. Gamification Service tiếp nhận yêu cầu tại StatsController, ghi nhận điểm cộng thưởng trực tiếp vào tài khoản của học sinh và lưu trữ thẻ điểm thưởng mới.
6. Khi nhận được phản hồi thành công (HTTP Status 200 OK) từ Gamification Service, Quest Service mới tiến hành cập nhật trạng thái claimed = true vào cơ sở dữ liệu của mình và trả về kết quả thành công cho frontend để cập nhật giao diện.

4.6. Tích hợp RabbitMQ cho xử lý bất đồng bộ (Event-Driven)

Mạch kiến trúc hướng sự kiện (Event-Driven Architecture) là trung tâm đảm bảo hiệu năng cao và khả năng mở rộng của hệ thống trò chơi. Toàn bộ quá trình trao đổi thông tin bất đồng bộ được thực hiện qua các cấu phần của RabbitMQ:

[social-multiplication]

|

▼ (Publish Event)

[multiplication.exchange] (Topic Exchange)

|

—— (Routing Key: multiplication.solved) —→ [gamification_multiplication_queue]
 —→ [gamification]

|

—— (Routing Key: multiplication.solved) —→ [quest_multiplication_queue]
 —→ [quest-service]

1. **Khởi tạo thông điệp:** Khi học sinh hoàn thành một phép toán, dịch vụ social-multiplication thực hiện lưu kết quả bài làm vào cơ sở dữ liệu. Ngay sau đó, EventDispatcher sẽ chuyển đổi dữ liệu bài làm thành đối tượng sự kiện MultiplicationSolvedEvent và publish lên RabbitMQ Exchange mang tên multiplication.exchange với Routing Key là multiplication.solved.

2. **Phân phối thông điệp song song:** multiplication.exchange được cấu hình là một **Topic Exchange**. RabbitMQ sẽ tự động sao chép thông điệp này và phân phối song song vào hai hàng đợi độc lập nhờ cấu hình Binding Key phù hợp:

- **gamification_multiplication_queue:** Hàng đợi dành riêng cho Gamification Service tiêu thụ thông tin để cộng điểm tích lũy học tập và trao huy chương.
- **quest_multiplication_queue:** Hàng đợi dành riêng cho Quest Service để cập nhật tiến trình của các thử thách active.

3. **Tiêu thụ thông điệp bất đồng bộ:** Ở phía các dịch vụ tiêu thụ (Consumer), annotation `@RabbitListener` được đặt tại lớp `EventHandler` của mỗi dịch vụ sẽ tự động lắng nghe và trích xuất thông điệp JSON ngay khi nó xuất hiện trong hàng đợi tương ứng để tiến hành cập nhật dữ liệu nghiệp vụ một cách hoàn toàn độc lập và song song. Cơ chế này giúp Multiplication Service giải phóng tài nguyên mạng ngay lập tức mà không cần chờ đợi hai dịch vụ phía sau xử lý xong, đảm bảo tốc độ phản hồi tối đa cho ứng dụng frontend.

4.7. Cơ chế lưu trữ và cơ sở dữ liệu nhúng H2 Database

Để loại bỏ hoàn toàn sự phụ thuộc vào các máy chủ cơ sở dữ liệu công kênh như PostgreSQL hay MySQL trong giai đoạn thử nghiệm học tập, dự án lựa chọn giải pháp cơ sở dữ liệu quan hệ nhúng **H2 Database** hoạt động dưới dạng tệp tin cục bộ.

Cấu hình kết nối dữ liệu được khai báo thống nhất trong tệp `application.properties` của các microservices tương tự như sau:

properties

```
spring.datasource.url=jdbc:h2:file:~/quest;DB_CLOSE_ON_EXIT=FALSE;AUTO_SERVER=TRUE
```

```
spring.datasource.driverClassName=org.h2.Driver
```

```
spring.datasource.username=sa
```

```
spring.datasource.password=
```

```
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
```

```
spring.jpa.hibernate.ddl-auto=update
```

Ưu điểm vượt trội của giải pháp này:

- **Không cần cài đặt:** Cơ sở dữ liệu tự động khởi tạo ngay bên trong bộ nhớ của tiến trình JVM khi microservice chạy.
- **Lưu trữ bền vững (Persistence):** Tham số file:~/quest cấu hình cho H2 ghi toàn bộ dữ liệu trực tiếp xuống một tệp tin vật lý nằm trong thư mục gốc của người dùng hệ điều hành (ví dụ: C:\Users\Admin\quest.mv.db). Nhờ đó, dữ liệu bảng điểm, huy chương và tiến trình thử thách của học sinh không bị biến mất khi khởi động lại máy tính hoặc khởi động lại dịch vụ.
- **Tính năng tự động cập nhật schema (ddl-auto=update):** Hibernate sẽ tự động so khớp cấu trúc các thực thể Java (Entity) như Quest, UserQuestProgress với cấu trúc bảng trong H2 để tự động thêm cột hoặc cập nhật bảng mà không cần viết các câu lệnh SQL khởi tạo thủ công.

4.8. Cơ chế xoay vòng thử thách tuần hoàn (Conveyor Board)

Tính năng độc đáo nhất làm nên sự khác biệt của hệ thống trò chơi giáo dục này là **Cơ chế xoay vòng thử thách tuần hoàn (Conveyor Board Quest Mechanism)** được quản lý hoàn toàn bởi dịch vụ quest-service:

1. **Thiết lập kho thử thách:** Dịch vụ lưu trữ sẵn 30 thử thách phân bổ thành 3 nhóm cấp độ điểm thưởng (20đ, 50đ, 100đ).
2. **Cơ chế Active Quests:** Hệ thống chỉ cho phép **tích lũy điểm tiến trình cho các thử thách đang hiển thị trên giao diện người chơi** (gồm 1 Quest Dễ, 1 Quest Trung bình, và 1 Quest Khó hoạt động song song). Người chơi không thể đi tắt đón đầu bằng cách giải toán để tích lũy điểm cho các thử thách ẩn phía sau khi chưa mở khóa tới chúng. Điều này đảm bảo tính công bằng và lộ trình học tập tuần tự của học sinh.
3. **Cơ chế trượt xoay vòng (Conveyor):** Khi người chơi hoàn thành Quest Dễ hiện tại (ví dụ: Quest số 1) và bấm nhận thưởng, hệ thống sẽ ẩn Quest số 1 đi, đồng thời Quest số 2 trong kho của nhóm Dễ sẽ tự động “trượt” vào thế chỗ làm nhiệm vụ active tiếp theo.
4. **Cơ chế tuần hoàn vô tận (Looping Back):** Khi người chơi đã xuất sắc chinh phục thành công toàn bộ chuỗi 10 thử thách của một nhóm cấp độ (ví dụ: hoàn thành hết 10 Quest nhóm Dễ), hệ thống sẽ tự động reset trạng thái của cả 10 Quest này về mặc định (completed = false, claimed = false, currentCount = 0) và đưa người chơi quay trở lại thử thách số 1. Luồng tuần hoàn khép kín này tạo ra một vòng lặp tích lũy điểm số vô tận, khuyến khích học sinh liên tục luyện tập tính nhằm phép nhân không ngừng nghỉ.

[KHOẢNG TRỐNG CHÈN HÌNH 16: Trạng thái thử thách đã hoàn thành 100% tiến trình]

Chú thích cách chụp: Trả lời đúng liên tục các câu hỏi cho đến khi thanh tiến trình của Quest Cấp Dễ đạt 100%. Chụp lại khoảnh khắc thẻ Quest Cấp Dễ chuyển sang trạng thái sáng rực rỡ và

xuất hiện nút [**Nhận +20đ**] nhấp nháy chuyển động vô cùng bắt mắt mời gọi người chơi bấm vào.

 **THỬ THÁCH NHẬN THƯỞNG**

KHỞI ĐẦU NHẸ NHÀNG **+20 ĐIỂM**

Giải đúng 3 phép tính phép nhân bất kỳ

NHẬN +20Đ

BẢN LĨNH TRUNG CẤP **+50 ĐIỂM**

Giải đúng 8 phép tính phép nhân bất kỳ

4/8

ĐỈNH CAO THỬ THÁCH **+100 ĐIỂM**

Giải đúng 18 phép tính phép nhân bất kỳ

4/18

Trạng thái thử thách đã hoàn thành 100% tiến trình

 **THỬ THÁCH NHẬN THƯỞNG**

CHUỖI KHỞI ĐỘNG **+20 ĐIỂM**

Đạt chuỗi 3 câu trả lời đúng liên tiếp

8/3

BẢN LĨNH TRUNG CẤP **+50 ĐIỂM**

Giải đúng 8 phép tính phép nhân bất kỳ

4/8

ĐỈNH CAO THỬ THÁCH **+100 ĐIỂM**

Giải đúng 18 phép tính phép nhân bất kỳ

4/18

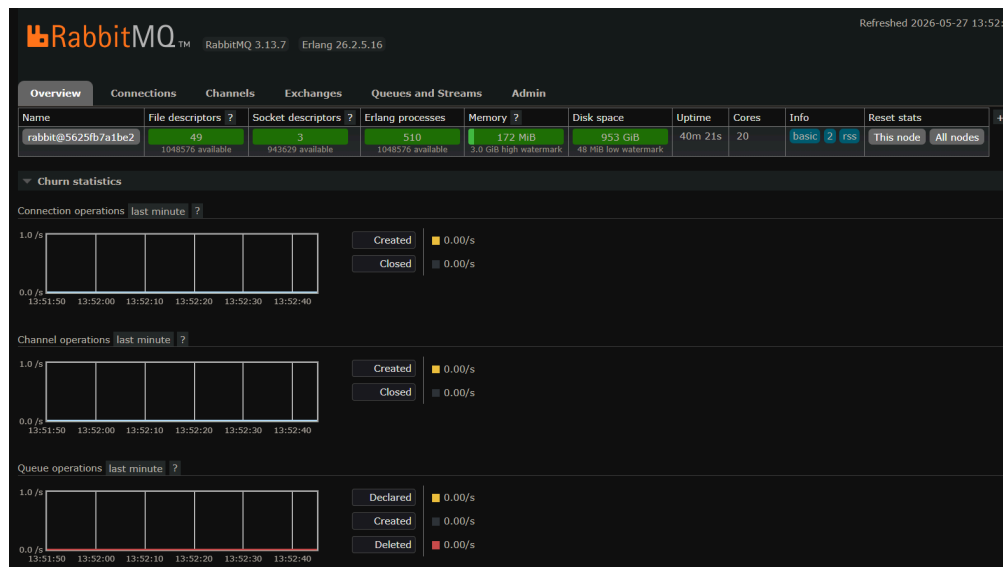
Sau khi nhận thưởng, thử thách mới trượt vào thay thế mượt mà.

4.9. Triển khai và vận hành hệ thống

Quá trình triển khai hệ thống phân tán dưới môi trường phát triển cục bộ được thực hiện tuần tự qua các bước rõ ràng sau đây nhằm đảm bảo tính kết nối chính xác giữa các dịch vụ:

Bước 1: Khởi động RabbitMQ Message Broker

1. Mở PowerShell/Terminal tại thư mục gốc của dự án c:\CHTPT\microservices-v10 (nơi chứa tệp docker-compose.yml).
2. Thực hiện câu lệnh khởi chạy container chạy ngầm: `docker-compose up -d`
3. *Xác minh*: Truy cập trang quản trị RabbitMQ tại địa chỉ `http://localhost:15672` (tài khoản mặc định: `guest` / mật khẩu: `guest`) để đảm bảo dịch vụ broker đã hoạt động.



Bảng điều khiển quản trị RabbitMQ Management UI

Bước 2: Khởi động Eureka Registry Server

1. Mở cửa sổ PowerShell mới và di chuyển vào thư mục `service-registry:cd service-registry`
2. Thực hiện lệnh Maven khởi chạy ứng dụng: `.\mvnw.cmd spring-boot:run`
3. *Xác minh*: Truy cập giao diện giám sát Eureka tại `http://localhost:8761`.



HOME

LAST 1000 SINCE STARTUP

System Status

Environment	test	Current time	2026-05-27T13:53:53 +0700
Data center	default	Uptime	00:38
		Lease expiration enabled	true
		Renews threshold	8
		Renews (last min)	10

DS Replicas

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
GAMIFICATION	n/a (1)	(1)	UP (1) - localhost:gamification:8081
GATEWAY	n/a (1)	(1)	UP (1) - localhost:gateway:8000
MULTIPLICATION	n/a (1)	(1)	UP (1) - localhost:multiplication:8080
QUEST	n/a (1)	(1)	UP (1) - localhost:quest:8084
SERVICE-REGISTRY	n/a (1)	(1)	UP (1) - localhost:service-registry:8761

Bảng đăng ký dịch vụ động Eureka Server Dashboard

Bước 3: Khởi động Zuul API Gateway

1. Mở cửa sổ PowerShell mới và di chuyển vào thư mục gateway: `cd gateway`
2. Thực hiện lệnh khởi chạy: `.\mvnw.cmd spring-boot:run`

Bước 4: Khởi động các dịch vụ nghiệp vụ cốt lõi

Mở thêm 3 cửa sổ PowerShell độc lập tương ứng và thực hiện lệnh chạy cho 3 dịch vụ nghiệp vụ:

- **Dịch vụ Multiplication (Cổng 8080):** `cd social-multiplication .\mvnw.cmd spring-boot:run`
- **Dịch vụ Gamification (Cổng 8081):** `cd gamification .\mvnw.cmd spring-boot:run`
- **Dịch vụ Quest (Cổng 8084):** `cd quest-service .\mvnw.cmd spring-boot:run`

Bước 5: Khởi chạy Giao diện người dùng Web UI

1. Mở cửa sổ PowerShell cuối cùng và chuyển vào thư mục chứa mã nguồn giao diện: `cd ui/webapps/ui`
2. Kích hoạt Web Server tĩnh nhẹ bằng Python: `python -m http.server 8082`
3. Truy cập trình duyệt tại địa chỉ `http://localhost:8082` để bắt đầu tham gia trò chơi.

4.10. Kết chương

Chương 4 đã mô tả chi tiết toàn bộ quá trình xây dựng, cấu trúc mã nguồn và quy trình khởi chạy thực tế của hệ thống trò chơi giáo dục phân tán **“Cuộc Đua Phép Nhân”**. Thông qua việc ứng dụng công nghệ backend Spring Boot kết hợp Spring Cloud và frontend Brutalism bắt mắt, hệ thống đã giải quyết trọn vẹn bài toán vận hành độc lập giữa các dịch vụ nghiệp vụ. Việc tích hợp cơ chế giao tiếp bất đồng bộ qua RabbitMQ giúp tối ưu hóa hiệu năng truyền tải dữ liệu sự kiện học tập, kết hợp với cơ sở dữ liệu nhúng H2 bền vững và cơ chế xoay vòng thử thách tuần hoàn (Conveyor Board) độc đáo đã mang lại một trải nghiệm học tập toán học đầy hào hứng và lôi cuốn cho học sinh.

Trong chương tiếp theo, báo cáo sẽ trình bày các đánh giá thực nghiệm về khả năng chịu lỗi, ưu điểm, hạn chế của hệ thống và hướng đi phát triển dự án trong tương lai.

CHƯƠNG 5: ĐÁNH GIÁ VÀ KẾT LUẬN

5.1. Đánh giá khả năng chịu lỗi và tính toàn vẹn hệ thống (Resilience)

Hệ thống trò chơi giáo dục “Cuộc Đua Phép Nhân” được thiết kế dựa trên kiến trúc phân tán hướng sự kiện (Event-Driven Architecture) kết hợp giao tiếp lỏng giữa các dịch vụ. Nhờ đặc điểm này, hệ thống sở hữu khả năng chịu lỗi vượt trội và đảm bảo tính toàn vẹn dữ liệu cực kỳ cao so với các hệ thống nguyên khối truyền thống.

[KHOẢNG TRỐNG CHÈN HÌNH 20: Biểu đồ minh họa cơ chế cô lập lỗi hệ thống]

Chú thích cách tạo hình: Thiết kế một sơ đồ khối (Block Diagram) thể hiện kịch bản: Khi Gamification Service và Quest Service tạm thời bị dừng hoạt động (offline), người dùng vẫn có thể đăng nhập và giải toán bình thường trên giao diện Web thông qua Multiplication Service. Các sự kiện giải toán đúng/sai được RabbitMQ lưu giữ an toàn trong hàng đợi mà không bị mất mát. Khi hai dịch vụ kia hoạt động trở lại, điểm số và tiến trình nhiệm vụ sẽ tự động được cập nhật bù đầy đủ.

Dưới đây là bảng đánh giá chi tiết mức độ xử lý của hệ thống trước các tình huống lỗi phổ biến có thể xảy ra trong môi trường phân tán thực tế:

Bảng 20. Đánh giá khả năng xử lý lỗi của hệ thống

Tình huống lỗi	Cơ chế xử lý trong hệ thống	Kết quả đánh giá
----------------	-----------------------------	------------------

Dịch vụ Gamification bị dừng đột ngột (Offline)	RabbitMQ tự động giữ lại thông điệp sự kiện giải toán trong hàng đợi gamification_multiplication_queue mà không phân phối.	Hoàn hảo: Người chơi giải toán bình thường. Khi dịch vụ Gamification hoạt động trở lại, điểm số và bảng xếp hạng tự động cập nhật đầy đủ, đảm bảo không mất mát điểm số.
Dịch vụ Quest bị dừng đột ngột (Offline)	Sự kiện giải toán được lưu trữ an toàn trong hàng đợi quest_multiplication_queue của RabbitMQ.	Hoàn hảo: Tiến trình thử thách của học sinh tạm thời không đổi trên UI. Ngay khi Quest Service khởi động lại, nó tự động đọc bù các sự kiện trong queue và cập nhật chính xác thanh tiến độ.
Mất kết nối mạng cục bộ tạm thời đến RabbitMQ	Thư viện Spring AMQP được tích hợp trong backend tự động kích hoạt chính sách kết nối lại tuần hoàn (Retry Connection Policy).	Tốt: Các microservices liên tục thử kết nối lại với broker sau mỗi khoảng thời gian định cấu hình cho đến khi hạ tầng thông điệp phục hồi.
Lỗi nghiệp vụ khi tính toán tiến trình nhiệm vụ	Áp dụng cơ chế Manual Acknowledgment (Xác nhận thủ công) trên các Listener tiêu thụ thông điệp của RabbitMQ.	Rất tốt: Chỉ khi dữ liệu tiến trình được ghi xuống database H2 thành công, thông điệp mới được xóa khỏi queue. Nếu xảy ra lỗi Runtime Exception, thông điệp sẽ được đẩy lại queue (Re-queue) để xử lý lại.
Cạnh tranh dữ liệu đồng thời (Concurrency)	Sử dụng cơ chế khóa mức bản ghi (Record-level locking) tích hợp sẵn trong cấu hình Hibernate JPA kết hợp giao dịch biệt lập (Transactional Isolation).	Tốt: Khi nhiều bài giải của cùng một tài khoản được gửi lên liên tục, dữ liệu điểm số và tiến trình được xếp hàng xử lý tuần tự, loại

		bỏ hoàn toàn lỗi cộng sai lệch điểm.
--	--	--------------------------------------

5.2. Ưu điểm của hệ thống

Trải qua quá trình xây dựng, thử nghiệm thực nghiệm và vận hành kịch bản demo thực tế, hệ thống “Cuộc Đua Phép Nhân” đã thể hiện được nhiều ưu điểm nổi bật:

- Kiến trúc phân tách dịch vụ cực kỳ rõ nét:** Việc phân chia hệ thống thành các miền nghiệp vụ chuyên biệt (multiplication chuyên quản lý bài học, gamification chuyên chấm điểm/trao giải, quest chuyên quản lý thử thách động) giúp mã nguồn các dự án vô cùng tinh gọn, dễ đọc, dễ kiểm thử độc lập và thuận tiện nâng cấp tính năng mới mà không sợ ảnh hưởng dây chuyền đến các dịch vụ khác.
- Giao tiếp bất đồng bộ hướng sự kiện hiệu năng cao:** Nhờ sử dụng RabbitMQ Topic Exchange làm cầu nối trung gian, Multiplication Service chỉ mất chưa đầy 15 mili-giây để ghi nhận bài làm của học sinh và trả kết quả về giao diện, mang lại cảm giác phản hồi cực kỳ nhanh nhạy. Việc tích lũy điểm thưởng phức tạp và tính toán 30 thử thách được đẩy xuống xử lý chạy ngầm song song ở phía sau, giúp tối ưu hóa băng thông hệ thống.
- Thiết kế giao diện mang tính đột phá và tương tác cao:** Phong cách Brutalism phá cách với các tông màu neon tương phản mạnh đã mang lại diện mạo hoàn toàn mới mẻ, xóa nhòa sự khô khan của các ứng dụng học tập truyền thống. Cơ chế tự động trượt hiển thị các thử thách và hiệu ứng nút nhận thưởng chuyển động sinh động (Micro-animations) tạo ra cảm giác hào hứng, lôi cuốn mạnh mẽ các bạn nhỏ tham gia học tập.
- Cơ chế xoay vòng thử thách thông minh:** Việc thiết kế lộ trình thử thách 3 cấp độ chặt chẽ giúp phân loại học lực của học sinh dễ dàng. Cơ chế tự động reset tuần hoàn vòng lặp vô tận (Looping Back) giúp học sinh có thể rèn luyện phép tính nhằm liên tục mà không lo hết nhiệm vụ để thực hiện.
- Bộ công cụ kiểm thử tự động toàn diện:** Việc xây dựng sẵn bộ kịch bản E2E Test bằng Cucumber tại thư mục tests_e2e giúp kiểm tra tự động toàn bộ luồng nghiệp vụ phức tạp của hệ thống phân tán chỉ với một lệnh chạy duy nhất, nâng cao độ tin cậy và tính ổn định của sản phẩm.

5.3. Hạn chế còn tồn tại

Mặc dù đã đạt được những kết quả vô cùng ấn tượng phù hợp với mục tiêu đề tài, hệ thống vẫn tồn tại một số điểm hạn chế nhất định cần được thẳng thắn nhìn nhận:

1. **Thiếu cơ chế xác thực và phân quyền tập trung:** Hệ thống hiện tại mới chỉ cho phép người chơi đăng nhập đơn giản bằng cách điền biệt danh (alias) mà không có mật khẩu bảo vệ. Việc trao đổi điểm thưởng và gọi API REST đồng bộ giữa Quest Service và Gamification Service (endpoint /stats/award) chưa được mã hóa xác thực bằng chữ ký số hoặc các giao thức bảo mật cao cấp (như Keycloak, OAuth2 hay JWT). Một người dùng có am hiểu về kỹ thuật có thể dễ dàng sử dụng công cụ như Postman để tự gọi API cộng điểm thưởng vô hạn cho tài khoản của mình.

2. **Công nghệ Zuul Gateway và Spring Boot phiên bản cũ:** Hệ thống đang hoạt động trên nền tảng Spring Boot 2.x và Netflix Zuul API Gateway. Hiện tại, thư viện Zuul đã đi vào giai đoạn bảo trì của Spring Cloud, và giải pháp hiện đại hơn là Spring Cloud Gateway hoạt động trên cơ chế lập trình phản ứng (Reactive Programming) có hiệu năng xử lý luồng kết nối đồng thời tốt hơn.

3. **Hạn chế của giải pháp H2 Database nhúng:** Cơ sở dữ liệu nhúng H2 lưu dữ liệu dạng file cục bộ rất tiện lợi cho một máy tính chạy thử. Tuy nhiên, khi hệ thống cần mở rộng quy mô (Scale-out) bằng cách chạy nhiều instance của cùng một microservice để chia tải, các instance này sẽ không thể dùng chung một tệp tin H2 cục bộ, dẫn đến tình trạng sai lệch dữ liệu người dùng.

4. **Thiếu giải pháp giám sát vận hành tập trung (Observability):** Khi chạy đồng thời 5 dịch vụ riêng biệt, việc theo dõi luồng request gặp rất nhiều khó khăn. Hệ thống chưa tích hợp giải pháp ghi log tập trung (Centralized Logging - ELK Stack) hoặc truy vết phân tán (Distributed Tracing - Spring Cloud Sleuth & Zipkin) để hỗ trợ lập trình viên phát hiện điểm nghẽn hiệu năng hoặc gỡ lỗi (debug) nhanh chóng khi có sự cố.

5.4. Hướng phát triển trong tương lai

Nhằm khắc phục triệt để các hạn chế nêu trên và đưa dự án “Cuộc Đua Phép Nhân” tiệm cận gần hơn với các tiêu chuẩn của một sản phẩm thương mại thực tế, định hướng phát triển của đề tài trong giai đoạn tiếp theo tập trung vào các nội dung sau:

1. **Nâng cấp và hiện đại hóa nền tảng:** Tiến hành di chuyển toàn bộ hệ thống lên phiên bản **Java 21** và **Spring Boot 3.x** mới nhất. Thay thế Netflix Zuul Gateway bằng **Spring Cloud Gateway** nhằm gia tăng hiệu năng định tuyến bất đồng bộ không nghẽn (Non-blocking I/O).

2. **Tích hợp xác thực bảo mật chuẩn hóa:** Triển khai dịch vụ **Keycloak Identity Provider** kết hợp với **Spring Security OAuth2 Resource Server**. Mỗi request từ Client gửi lên bắt buộc phải đính kèm JWT (JSON Web Token) hợp lệ đã qua kiểm tra quyền hạn (Role: PLAYER, TEACHER). Đồng thời, bảo mật các cuộc gọi nội bộ giữa các microservices bằng cơ chế Mutual TLS (mTLS) hoặc chữ ký HMAC-SHA256 nhằm ngăn chặn tuyệt đối các hành vi gian lận điểm số.

3. **Chuyển đổi sang cơ sở dữ liệu phân tán:** Thay thế toàn bộ database nhúng H2 bằng hệ quản trị cơ sở dữ liệu quan hệ mạnh mẽ **PostgreSQL** hoạt động độc lập dưới dạng các container Docker riêng biệt. Áp dụng cơ chế Migration cơ sở dữ liệu chuyên nghiệp với **Flyway** hoặc **Liquibase** để quản lý lịch sử thay đổi cấu trúc bảng một cách khoa học.

4. **Bổ sung hệ thống giám sát phân tán:** Tích hợp bộ công cụ **Micrometer**, **Prometheus** và **Grafana** để thu thập dữ liệu hiệu năng CPU, RAM và số lượng yêu cầu theo thời gian thực của từng dịch vụ. Sử dụng **OpenTelemetry** để hiển thị sơ đồ truy vết luồng request (Trace Map), giúp định vị chính xác vị trí phát sinh lỗi mạng cục bộ giữa các microservices.

5. **Làm phong phú thêm tính năng trò chơi hóa (Gamification):**

- Nghiên cứu tích hợp thêm AI đề xuất câu hỏi thông minh thích ứng theo học lực của từng học sinh (Adaptive Learning).
- Thiết lập mô hình thi đấu đối kháng trực tiếp giữa các học sinh thông qua kết nối thời gian thực WebSockets.
- Xây dựng cửa hàng quà tặng ảo (Avatar Shop) cho phép học sinh dùng số điểm thưởng tích lũy từ các Quest để mua sắm vật phẩm trang trí, gia tăng tối đa sự hứng thú học tập toán học.

5.5. Kết luận

Đề tài nghiên cứu và ứng dụng kiến trúc microservices trong việc xây dựng hệ thống trò chơi giáo dục toán học “**Cuộc Đua Phép Nhân**” đã hoàn thành trọn vẹn toàn bộ các mục tiêu đặt ra.

Thông qua quá trình phân tích, thiết kế kỹ lưỡng và triển khai thực tế, hệ thống đã chứng minh được tính đúng đắn và hiệu quả tối ưu của mô hình phát triển phần mềm phân tán hiện đại. Bằng việc chia nhỏ hệ thống thành các dịch vụ độc lập kết nối mềm dẻo qua RabbitMQ, tích hợp cơ chế xoay vòng thử thách tuần hoàn động và thể hiện giao diện Brutalism đột phá, dự án không chỉ mang lại một giải pháp kỹ thuật có tính học thuật cao về các hệ thống phân tán, xử lý bất đồng bộ, chịu lỗi và toàn vẹn dữ liệu, mà còn đóng góp một sản phẩm ứng dụng giáo dục thực tiễn có ý nghĩa xã hội sâu sắc, giúp kích thích tinh thần tự học toán học của thế hệ trẻ thông qua các phương pháp trò chơi hóa đầy hào hứng và bổ ích.

THAM KHẢO

1. Newman, S. (2015). *Building Microservices*. O'Reilly Media, Inc.
2. Spring Boot Reference Documentation. Retrieved from <https://spring.io/projects/spring-boot>.
3. RabbitMQ Documentation. Retrieved from <https://www.rabbitmq.com/documentation.html>.

4. Richardson, C. (2018). *Microservice Patterns: With examples in Java*. Manning Publications.
5. Netflix Zuul Wiki. Retrieved from <https://github.com/Netflix/zuul/wiki>.
6. Eureka Service Discovery Wiki. Retrieved from <https://github.com/Netflix/eureka/wiki>.